

**GUARDRAILS MICROSERVICE: A FIVE-LAYER
VALIDATION ENSEMBLE FOR RELIABLE
RETRIEVAL-AUGMENTED GENERATION IN
HIGH-STAKES FINANCIAL WORKFLOWS**

25-26J-129

Final Report

Wijetunga W.L.A.T.

BSc (Hons) in Information Technology Specializing in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

**GUARDRAILS MICROSERVICE: A FIVE-LAYER
VALIDATION ENSEMBLE FOR RELIABLE
RETRIEVAL-AUGMENTED GENERATION IN
HIGH-STAKES FINANCIAL WORKFLOWS**

25-26J-129

Final Report

BSc (Hons) in Information Technology Specializing in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

It is hereby declared that this is an original piece of work, and no part of this dissertation contains any material which has been submitted at any other university for a degree or diploma elsewhere, to the best of my knowledge. It is further declared that this dissertation does not contain any material published elsewhere or written by any other individual, except when such references are acknowledged within the text of the dissertation.

In addition to this, I hereby grant the non-exclusive right to the Sri Lanka Institute of Information Technology to reproduce and publish my dissertation wholly or partly in printed or electronic form.

Name	Student ID	Signature
Wijetunga W.L.A.T.	IT22	

The supervisor/s should certify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

.....
Signature of the Supervisor:

.....
Date

.....
Signature of the Co Supervisor:

.....
Date

ABSTRACT

The use of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) pipelines on document-intensive tasks like bank statements analysis, invoice processing, and purchase order auditing is profoundly transforming the financial sector. But this change also poses a severe and unresolved crisis: the artificially hallucinating phenomenon, the linguistically speech like and structurally realistic but factually false and mathematically unsound productions. Even one spectrally catastrophic hallucination can in high stakes financial settings lead to regulatory non-compliance, false financial reporting, and a structural collapse of institutional trust.

This dissertation describes the design, implementation and rigorous empirical testing of a production-scale Guardrails Microservice that runs as a deterministic safety kernel between any probabilistic RAG pipeline and downstream financial systems. The microservice has a five-layer sequential validation ensemble, consisting of Schema Adherence (through Pydantic v2), Internal Consistency (arithmetic cross-checks), Domain Rule Enforcement (configurable business-logic heuristics), PII Leakage Protection (via Microsoft Presidio), and Semantic Faithfulness scoring (via SentenceTransformers all-MiniLM-L6-v2 at a configurable 0.78 threshold).

The system is designed in the Human-Near-the-Loop (HNTL) paradigm, which allows routes to be quantitatively and reliably routed: high-confidence routes are automatically processed, and anomalies and low-confidence routes are automatically escalated into special human oversight queues, without compromising regulatory compliance.

It is shown on an adversarial edge-case set (n=400) and a synthetic dataset derived on a FinanceBench (n=1,200) that the five-layer ensemble raises system Precision by 0.42 to 0.94, F1-score by 0.50 to 0.955 and Faithfulness by 0.64 to 0.93, while maintaining sub-300ms latency. The layer-wise failure analysis shows that Internal Consistency is the most common failure mode in LLMs that handle financial documents (31.0% of all adversarial rejections), and that other layers such as Schema Adherence (21.8%), Domain Rules (24.0%), PII Leakage (10.8%), and Semantic Faithfulness (12.5%) offer complementary defense. All these results confirm the architecture as a viable, production-ready safety kernel to enterprise FinAI systems.

Keywords:

Hallucination Detection, Retrieval-Augmented Generation, FinAI, Guardrails Microservice, Human-Near-the-Loop (HNTL), Design Science Research, PII Leakage, Semantic Faithfulness.

ACKNOWLEDGEMENT

I would also like to say a big thank you to my dissertation supervisors in the Sri Lanka Institute of Information Technology who have provided me with invaluable guidance, constructive feedback, and unhindered support throughout this fourth-year research project. Their background in the study of artificial intelligence and enterprise system greatly influenced the direction and quality of this work.

I am also most appreciative of the academic staff of the Department of Information Technology who taught me during my undergraduate programme and whose teaching gave me the basis of theory and practice through which this research was constructed. I want to also mention special thanks to my colleagues and peers as well because of their support, intellectual debates, and cooperation in the research process.

I owe my family my utmost gratitude and their non-conditional support, patience, and encouragement in what was a rather intense and demanding research period. They believed in me to accomplish this project and this proved to be a motivating factor.

Disclosure Statement AI: This dissertation was written by Wijetunga W.L.A.T. (Thenuka). Grok (xAI) was exclusively applied to refine the text, enlarge its structure, academic wording, and drafting with human oversight and review. Everything that is technically implemented, code architecture, data models, experimental design and core intellectual contribution is solely of the author. The codebase of the Guardrails Microservice (v1.0.0) is original work created by the author.

Table of Contents

DECLARATION	3
ABSTRACT	4
ACKNOWLEDGEMENT	5
LIST OF FIGURES	8
LIST OF TABLES	8
LIST OF ABBREVIATIONS	8
1. INTRODUCTION	9
1.1 Background and Literature Survey	9
1.1.1 The evolution of automation in financial and enterprise services	9
1.1.2 Mechanics and adoption of Retrieval-Augmented Generation (RAG)	10
1.1.3 The taxonomy of artificial hallucinations in Generative AI	11
1.1.4 The hallucination crisis in high-stakes domains	13
1.1.5 The regulatory landscape and compliance requirements (2024–2026)	14
1.1.6 Existing mitigation strategies and their inherent limitations	15
1.1.7 Human-Near-the-Loop (HNTL) as a socio-technical solution	17
1.2 Research Gap	19
1.2.1 Theoretical Framework	19
1.3 Research Problem	21
1.4 Research Objectives	21
2. METHODOLOGY AND ARCHITECTURE	23
2.1 Methodology	23
2.1.1 System architecture and technology stack	24
2.1.2 Data modelling and input/output contracts	26
2.1.3 Layer 1: Schema adherence validation	27
2.1.4 Layer 2: Internal consistency and arithmetic grounding	28
2.1.5 Layer 3: Domain rule enforcement and anomaly detection	29
2.1.6 Layer 4: Context-aware PII leakage protection	31
2.1.7 Layer 5: Semantic faithfulness and hallucination detection	32
2.2 Commercialisation Aspects of the Product	35
2.3 Testing and Implementation	36
3. RESULTS AND DISCUSSION	39

3.1 Results	39
3.1.1 Overall performance metrics	40
3.2 Research Findings	42
3.2.1 Layer-wise failure analysis	42
3.3 Discussion	44
3.4 Ethical Considerations and Responsible AI	46
3.5 Comparative Analysis with Related Systems.....	47
3.6 Summary of Each Student's Contribution	49
4. CONCLUSIONS AND RECOMMENDATIONS	51
4.1 Conclusions	51
4.1.1 Summary of research and the Guardrails Microservice	51
4.1.2 Efficacy of the five-layer validation ensemble	52
4.1.3 Operationalising Human-Near-the-Loop (HNTL) and system latency	53
4.2 Recommendations and Future Work	54
REFERENCES	57
APPENDIX A: GUARDRAILS MICROSERVICE CODEBASE	63
APPENDIX B: EXAMPLE AUDIT LOGS AND VALIDATION OUTPUTS	66

LIST OF FIGURES

Figure 2.1: Standard RAG Pipeline Architecture highlighting points of failure	27
Figure 2.2: High-Level Architecture of the Guardrails Microservice as a middleware interceptor	28
Figure 2.3: Cosine Similarity Measurement for Semantic Faithfulness scoring	32
Figure 2.4: Endpoint request lifecycle and fail-fast sequence flow	34

LIST OF TABLES

Table 1 – Research Gap.....	15
Table 2 – Test Case 01	38
Table 3 – Test Case 02	39
Table 4 – Test Case 03	40
Table 5 – Test Case 04	41

LIST OF ABBREVIATIONS

1. INTRODUCTION

1.1 Background and Literature Survey

1.1.1 The evolution of automation in financial and enterprise services

Financial sector, which includes banking, investment, insurance, and enterprise administrative services have long taken a leading role in the globalization of digital transformation. This unyielding pursuit of operational efficiency is driven mostly by the sheer practicality of having to process, verify, reconcile and safely store enormous amounts of unstructured and semi-structured documentation daily. Over the last ten years, financial institutions have been using a mixture [1] of both traditional Robotic Process Automation (RPA) and Optical Character Recognition (OCR) technologies to retrieve key transactional data on bank statements, vendor invoices, clinical summaries, insurance claims, and purchase orders (Zhang et al., 2022).

Although these heritage systems were somewhat deterministically reliable, rule-based, they were inherently brittle. Any small change in the underlying template to a document, its spatial structure, its typography, or even a simple shift in the invoice format of a vendor often led to disastrous, complete extraction failures. These failures required constant and labour-intensive manual maintenance of templates and produced a lot of overhead in their operation, taking up resources that could be used to do other more valuable analytical work. Kaur and Singh (2023) emphasized the high expenses of sustaining these inflexible automation systems within the insurance industry, pointing out that the use of templates posed a major obstacle to flexibility and scalability.

The introduction of transformer-based Large Language Models (LLMs) was a radical change in the sphere of Document AI and intelligent automation. As compared to the traditional RPA systems, which can only work based on pre-defined syntactic rules, Generative AI models have a deep and contextual semantic comprehension of language. This semantic intelligence enables them to extract entities correctly, summarise complex financial stories, synthesise data based on cross-referenced appendices, map complex logical relationships of data, irrespective of the underlying spatial layout or formatting of a document (Bender et al., 2021). An LLM-powered system can, e.g., correctly find and extract a net payment value even when that payment is found in a non-standard location or in a strange language form, something

that no brittle, template-dependent RPA system can do at all.

Nevertheless, the underlying operational logic of LLMs: to predict the statistically most likely next token in a sequence using massive probability distributions trained on billions of training examples — poses substantial, unprecedented risks in high-stakes settings where deterministic accuracy is a non-negotiable condition. Since Bender et al. (2021) warned in their seminal debate about large language models as being stochastic parrots, they produce outputs in terms of statistical correlations and not any true understanding of truth or physical reality which has far-reaching and alarming consequences to the financial world in terms of application to mission-critical processes.

The conflict between the transformative efficiency benefits of modern LLMs and the high accuracy requirements of financial systems is the driving force behind the main problem of this dissertation. To resolve this tension, we need to go beyond mere faith in generative models, and create strong, deterministic engineering controls that may be used to verify, filter, and audit each and every AI-generated output with high confidence before it is allowed to drive into a production financial system.

1.1.2 Mechanics and adoption of Retrieval-Augmented Generation (RAG)

As a way to tapped the remarkable semantic strength of LLMs, at the same time reducing their notoriously and experimentally proven propensity to information creation, Lewis et al. (2020) made the Retrieval-Augmented Generation (RAG) architecture formal [5]. This work, which was a foundational work, directly faced the knowledge-cutoff problem of all standalone, closed-book LLMs, the inability to access or reason about information that is not present in their training data, by dynamically promptly injecting retrieved, highly relevant document chunks into the active prompt context of the model during inference.

In a typical enterprise RAG pipeline, proprietary and confidential documents, i.e. an enterprise secure storage of life insurance claims, financial bank statements, vendor invoices or patient medical records, are loaded into the system in an offline pre-processing phase. This stage involves breaking down documents into smaller semantically coherent units of text and embedding them in high-dimensional vectors with a specialised embedding model. They are subsequently indexed and stored in a special vector database, which makes similarity-based

retrieval efficient. A similarity search is then conducted with this vector database when a system query is then issued and the most contextually relevant text chunks are retrieved. These fragments are then literally prepended to the prompt of the LLM as a grounding context working memory.

The inherent significance of this architectural pattern is that it pushes the language model to produce responses grounded in the given, confirmed, factual context, not basing them only on the statistical tendencies of its internal, pre-trained parametric memory. The practical implication on the financial applications is radical: an invoice-driven system which uses RAG can be trained to respond to a question about a particular vendor invoice by simply retrieving the invoice pages in question and producing an extraction based on that particular document, not on a guess of what invoices usually do.

RAG adoption has been tremendous in the financial sector. According to Deloitte Insights (2024), financial institutions that implemented pipelines based on RAG-based document processing experienced a substantial decrease in costs related to manual processing and better extraction throughput. PwC (2025) also suggested that RAG is one of the financially most important applications of generative AI in financial services, specifically in the generation of regulatory compliance documentation, analysis of financial reports, and automated underwriting procedures. In spite of these profits, there was a report convergence between the two industries on the same negative issue: RAG in itself does not necessarily ensure factual integrity [6], and the implementation of such systems without sound validation and control mechanisms poses unacceptable operational and regulatory hazard.

1.1.3 The taxonomy of artificial hallucinations in Generative AI

Although the RAG architectural pattern dramatically enhances the factual grounding over standalone closed-book LLMs, recent empirical studies have uniformly and conclusively shown that retrieval is not, in itself, a sufficient condition to ensure factual integrity. One of the most intractable bottlenecks in the broad safe implementation of AI in financial services is the phenomena known as artificial hallucinations by Ji et al. (2023) [3] — linguistically fluent in isolation, syntactically sensible, and semantically plausible, but factually false, numerically inconsistent, or based entirely on a source material that can never be verified.

In their extensive and highly-cited survey of hallucination in natural language generation Ji et al. (2023) have offered a strict [3] and strongly-practically useful taxonomy of such errors, dividing them into two main and separate categories. Intrinsic hallucinations are generated when the output that is produced is directly opposite to data that is present clearly and explicitly in the source material presented in the prompt context. This arises, say, when an LLM pulls out of an invoice a vendor amount of \$12,500 where it is plain and clear that the amount is actually \$11,250. By contrast, extrinsic hallucinations are produced when the model produces plausible and completely unverified information that cannot be traced to or based on any part of the text being retrieved — say when a model spontaneously invents a tax rate, an account number, or a payment term that is not anywhere present in the text being accessed.

Maynez et al. (2020) also provided additional knowledge in this area [2] by studying faithfulness and factuality in abstractive summarisation and showing that even the most advanced models systematically generate outputs with unverified or contradictory information when asked to synthesise the complex documents. The discovery is especially concerning in financial settings, where the summarisation of a bank statement or the derivation of certain numbers out of a financial report require unquestionable, machine-checkable accuracy as opposed to an approximation.

Extrinsic hallucinations are especially and acutely hazardous in a financial and insurance context as the retrieved contexts injected into the LLM prompt may contain thick numerical tabularizations, highly cross-referenced appendices, multi-clause regulatory terms, and nested sub-totals that are both deeply semantically understood and arithmetically sound. The capability to ensure rigid numeric and logical consistency across long-form, information-intensive documents is one of the most notable and enduring sources of failure of modern transformer architectures as Huang et al. (2021) showed in their analysis of factual consistency in text generation.

Practical implications of this taxonomy are essential to system design. Since intrinsic and extrinsic hallucinations may both appear in a RAG pipeline then an effective safety mechanism must make use of more than a single, qualitatively different validation strategy it cannot be based on a single check. The system that checks only structural schema adherence can accept a numerically inconsistent output, and the system that checks only semantic similarity can fail to detect a structurally ill-formed or mathematically incorrect output. This

five-layered imperative is the principle design of the five-layer validation ensemble suggested in this dissertation.

1.1.4 The hallucination crisis in high-stakes domains

Generative AI workflows have extremely limited tolerance to error based on the field of application. Although a hallucinated narrative detail in a creative writing assignment is just a small aesthetic inconvenience, a hallucinated object, value, or logical connection in a financial report or clinical summary has dire and instant real-world impacts. A single misread claim on an insurance policy, a falsified line on a bank statement, an inaccurate dosage of medication in a clinical record, or a wrong tax calculation in a purchase order can quickly lead to regulatory breach, inaccurate financial reporting, disastrous capital misplacement, and un-repaired loss of institutional trust and brand name.

The alarming severity of this problem has been stringently measured by recent authoritative industry benchmarks. The FinanceBench benchmark (2024) of Patronus AI in particular tested the ability of the state-of-the-art large language models to answer questions based on real-world financial reports, such as annual reports, earnings calls, and SEC filings, correctly. The findings were alarming: even the most competent closed-source models could not answer realistic financial questions (with evidence grounding rigorously imposed) with a success rate of up to 81 percent. It is not a fringe failure rate; it is a systemic, structural weakness of existing-generation models of autonomous financial processing.

These findings were also supported in a study by Chen et al. (2023), who evaluated the use of LLMs to understand financial documents, and showed that the highest error rates were always produced by the complex and multi-hop financial QA tasks that required cross-referencing of several tables or financial periods. One of the most common and operationally risky failure modes in automated financial document processing, identified by their work to be specifically entity swapping, i.e. the model recognizing the correct value of something but placing it in the incorrect entity (such as the total value of a buyer being assigned to the vendor).

Zhang et al. (2022) have developed a specially suggestive and analytically accurate expression, a spectrally catastrophic errors, to characterize a certain and particularly dangerous sub-group of financial AI hallucination. Such errors are defined by their superficial

plausibility: an LLM may produce invoice total of \$45,000 with a syntactically correct structure of a JSON file, a valid vendor name and correctly composed line items, but the arithmetic is incorrect, because the sum of the line items in the invoice is actually only \$42,500. The spectrality of the error is that it does not show up under normal API formatting and schema validation tests; only a deterministic arithmetic test would detect it. It is this type of error that is explicitly designed to be intercepted by the Internal Consistency layer of the proposed Guardrails Microservice.

Moreover, both Zhang et al. (2022) in the case of AI in banking, and Kaur and Singh (2023) in the case of insurance, both found that the downstream effects of such mistakes, once they have landed in a production financial database, are usually much greater than the size of the initial mistake. One fake transaction entry will tamper with reconciliation processes, cause incorrect regulatory reporting, and may require costly and time-intensive manual audit to detect and correct.

1.1.5 The regulatory landscape and compliance requirements (2024–2026)

With the active competition between financial institutions to roll out AI-based document processing pipelines in the hope of achieving operational efficiencies, world regulators have proposed more strict and targeted compliance regimes aimed at consumer protection, safeguarding the integrity of the market, and institutional responsibility. This regulatory environment is not only a legal requirement, but a basic architectural prerequisite that should be a part of the design of any production-grade FinAI system.

The United States Securities and Exchange Commission (SEC) has greatly stepped up its examination of what it calls AI washing, meaning falsely bolstering AI abilities and utilizing generative systems without any firm, auditable, and provable risk handling (SEC, 2025). The SEC has clarified that institutions cannot just declare their systems as being AI-powered without showing how these systems can match the same level of accuracy, fairness, and explainability of the processes performed by humans.

Specifically and directly relevant to the study, FINRA Regulatory Notice 24-09 (FINRA, 2024) specifically and directly mandates firms that use Generative AI and Large Language Models in any financial process involving clients or internal processes to impose strict

auditable supervisory controls. This notice requires that any model outputs have to be verifiably fair, balanced, and not misleading, which cannot be achieved by a bare, unvalidated RAG pipeline that is run without a safety mechanism. This is a regulatory requirement that gives the direct institutional rationale of the proposed Guardrails Microservice in this dissertation.

At the same time and on the international level, the General Data Protection Regulation (GDPR) implemented throughout the European Union subjects the processing of Personally Identifiable Information (PII) to a set of strict requirements, punishable by financial fines of up to 4% of the yearly turnover (European Union, 2024). The Gramm-Leach-Bliley Act (GLBA) in the United States places similar federal requirements on the financial institutions, which entails the establishment of end-to-end security and confidentiality of customer financial data (FTC, 2025).

The European Banking Authority (2025), the International Organisation of Securities Commissions (IOSCO, 2024) and the Bank for International Settlement (BIS, 2024) have all issued complementary guidance on AI risk management in financial services internationally. Another overall framework, the OECD AI Principles (OECD, 2025), highlights the importance of robustness, transparency, and human oversight as inexchangeable principles of responsible AI implementation. The United Kingdom Financial Conduct Authority (2024) also issued guidance that an AI system deployed in regulated financial services must be explainable, auditable, and meaningfully human supervised; again, the latter directly translates to the HNTL paradigm, and the audit trail provided by the Guardrails Microservice.

All these converging regulatory requirements lead to a very simple and explicit demand: any organisation using RAG-based LLM in financial processes must deploy programmatic, auditable, and verifiable controls that can be shown to prevent the further spread of inaccurate, misleading or privacy-invasive outputs. The Guardrails Microservice is created and tested in this regulatory context.

1.1.6 Existing mitigation strategies and their inherent limitations

The AI community has suggested and implemented a wide variety of advanced approaches in order to solve the crisis of hallucinations in generative models. Both solutions have real

merits, but they can be characterized by serious architectural and practical constraints that do not allow them as independent solutions in the financial arena where stakes are high.

Fine-tuning or hallucination-aware training is the most studied method of mitigation. Song et al. (2024) introduced RAG-HAT (Retrieval-Augmented Generation Hallucination-Aware Tuning), a pipeline specifically designed to fine-tune language model weights to explicitly recognise and avoid ungrounded generation. The outcomes of their experiments showed that RAG-HAT could decrease the rate of hallucinations by a significant margin of up to 37%. Nevertheless, the method has significant downsides to the financial deployment of enterprises. Fine-tuning is computationally costly and needs a lot of GPU resources. In addition, Gosmar and Dahl (2025) observed that fine-tuned models can typically have poor generalised reasoning performance on tasks not included in the fine-tuning dataset — a phenomenon referred to as catastrophic forgetting. When a financial institution is dealing with various document types, a model trained on invoices can also have a negative impact on the performance of the model on bank statements.

The second type of mitigation strategies is called agentic frameworks. The multi-agent system suggested by Gosmar and Dahl (2025) is an OVON-based system where structured JSON messages are sent over the network between a group of multiple agents of the LLM, and the debate and consensus mechanism is used to refine and validate generated messages, iterating the process. Although conceptually simple, agentic approaches add significant latency overhead. With every extra LLM inference call, hundreds of milliseconds are added to the processing pipeline. In financial settings with high throughput that require processing thousands of documents per hour, this latency cost makes purely agentic validation strategies operationally infeasible.

An accumulating literature supports embedding-based semantic faithfulness scoring, which is a lightweight alternative, computationally. Li et al. (2022) and Zhang et al. (2022) tested the idea of using dense vector embeddings to perform computational verification of an equivocal sentence being semantically entailed by the source document, instead of using a second LLM to verify. Mathematically based tools based on the cosine similarity of the generated claim vector with the retrieved context vector, such as those based on SentenceTransformers all-MiniLM-L6-v2 as described by Reimers and Gurevych (2019), allow a mathematically-grounded approach to hallucination detection.

Chen et al. (2023) showed that faithfulness detection through embedding similarity can be highly accurate in detecting semantic hallucinations in machine translation problems. Ji et al. (2023) also reported that embedding-based methods [3] are one of the most feasible and computationally inexpensive strategies of hallucination detection in use. Nonetheless, semantic similarity scoring cannot be solely used to validate financial documents. An arithmetically incorrect but semantically similar payload, like an invoice with a believably-sounding total that is not the sum of its line items, would score above a faithfulness threshold and still have a spectrally catastrophic error. This is a limiting element that proves the need to take a multi-layer approach.

Rawte et al. (2023) reviewed a wide perspective of hallucination detection and mitigation measures and found that no single method offers complete protection and that multi-modal, multi-layers defensive designs are the most prospective approach to attaining stable AI implementation. This conclusion is a direct corroboration of the architectural philosophy of the five-layer ensemble that is introduced in this dissertation, which takes the benefits of schema validation, deterministic arithmetic checking, rule-based domain enforcement, NLP-based PII detection, and embedding-based semantic faithfulness and integrates them into a single, unified, high-performance middleware layer.

1.1.7 Human-Near-the-Loop (HNTL) as a socio-technical solution

The issue of AI implementation in high-stakes areas is not only a technical issue; it is also a socio-technical one. The issue of the extent to which human supervision is necessary, when and in what ways it is best supervised is as critical as the algorithmic question of how to identify errors. The paradigm of Human-Near-the-Loop (HNTL) proposed by Jackson and Pinto (2024) is an extension of socio-technical frameworks of AI deployment, including the more traditional Human-in-the-Loop (HITL) and the more autonomous Human-on-the-Loop (HOTL) models, which are increasingly ineffective.

In a classic Human-in-the-Loop (HITL) system, each and every AI action or result must be explicitly approved by humans before being taken. Although this offers the greatest degree of control, it introduces a bottleneck of massive proportions and in many cases, a paralysing

factor in operations, completely nullifying the benefits of automation. The human approval of each and every extracted invoice or bank statement would demand a much larger workforce than the pre-automation benchmark in a high-volume financial document processing setting, and the technology is commercially impracticable (Amershi et al., 2019).

On the other hand, a completely automated Zero-Touch or Human-on-the-Loop (HOTL) system is completely based on the probabilistic outputs of the AI without a goal human intervention point. This practice is also unacceptable in the regulated financial settings as the hallucination crisis reveals. Even a small fraction of hallucinated financial extractions propagated through a HOTL system straight to production accounting databases can be found to violate regulation and lose money on scale.

As proposed by Jackson and Pinto (2024), HNTL paradigm is a middle ground that is both strategically intelligent and commercially viable. Having humans in an HNTL architecture brings episodic, high leverage control. High-confidence document processing flows that are routine can be allowed to run completely without human intervention. Nevertheless, the system always reserves the ability to recognize particular, targeted edge cases i.e. unusual, ambiguous or low-confidence payloads and only direct such anomalies to a human operator. This design philosophy maintains most of the efficiency of full automation, and ensures that the most risk-prone outputs are subjected to reasonable human review.

To make HNTL effective and reliable in a data architecture in an enterprise, it must have a robust, high-speed deterministic and programmatic confidence signal. Simply asking an LLM to rate its confidence is not enough; such self-assessments are inherently inaccurate and have been demonstrated to be systematically miscalibrated (Lin et al., 2021; OpenAI, 2023). The confidence signal has to be produced by another deterministic validation layer that checks the output with objective, mathematical and logical criteria. This is exactly what the Guardrails Microservice is engineered and developed to do: converting the probabilistic, subjective output of a RAG pipeline into a structured, quantitative and legally-auditable confidence score, which can be depended on to drive routing decisions in HNTL.

In his powerful vision of human-centred AI, Shneiderman (2020) claimed that believable AI systems should be reliable, safe, and trustworthy and that to accomplish this, it is necessary

to place a strong human control. The HNTL paradigm, as implemented by the Guardrails Microservice, offers just this: intensive, focused, and effective human control in the very places and times of greatest need, without the freezing out overhead of omnipresent HITL control.

1.2 Research Gap

Although the pace of development of Generative AI and its growing aggressive use in the financial sector is breathtaking and widespread, a gap in research is essential and has a significant impact on the enterprise implementation of RAG-based systems. Present production pipelines do not have a deterministic, programmatic safety kernel, that is, a lightweight, high-performance microservice that assesses the outputs of LLMs based on hard rules of software engineering, strict schema-validation, and deterministic mathematical analysis, as opposed to secondary probabilistic models or time-intensive agentic consensus mechanisms.

Although each element of the solution that is needed can be found in the literature on the topic, namely, semantic faithfulness scoring is described by Li et al. (2022) and Reimers and Gurevych (2019), PII detection is covered by the Presidio toolkit by Microsoft [12], and schema validation is a common practice in software engineering, no published study has ever proposed an integrated, production-ready, containerised microservice that orchestrates all five layers specifically for financial documents.

More importantly, no current solution provides all the following features together in a single, deployable system: (1) multi-layered, complementary validation, with deterministic and probabilistic checks; (2) sub-300ms processing latency with real-time enterprise workflows; (3) context-aware, document-type-sensitive PII detection; (4) structured, machine-readable output that can directly drive HNTL routing decisions. This dissertation fills this gap.

1.2.1 Theoretical Framework

The theoretical bases of this study are informed by three intersecting intellectual traditions: the Information Systems Design Science, the Socio-Technical Systems Theory, and the area

of AI Safety Engineering which is still developing. Every tradition will add a unique analytical prism to the design of the Guardrails Microservice and the explanation of its empirical findings.

The research is informed by the Design Science Research tradition (Hevner et al., 2004; Peffers et al., 2007) which assumes the basic epistemological orientation known as the view that rigorous contributions to academic knowledge in applied computing can be in the form of working artefacts (systems, models and methods) whose quality is established through an empirical process of evaluation against specified performance criteria. DSR justifies building the Guardrails Microservice as a piece of scholarly work, rather than as a software engineering deliverable, as long as the process of constructing it is well-documented, the evaluation methodology is rigorous, and the design decisions are based on and add to the overall body of theoretical knowledge.

Based on the Socio-Technical Systems Theory, this study borrowed the conceptual framework in understanding the HNTL paradigm as an automation decision much more than a mere automation decision. The theory of socio-technical systems, which is based on the studies of the Tavistock Institute and is thoroughly discussed in the literature of human-computer interaction by such researchers as Amershi et al. (2019) and Lee (2018), suggests that the best organisational systems are those that optimally joint-optimize the technical and social subsystems at the same time. A technicalist approach to mitigation of hallucinations — an approach that seeks to attain 100% automation via more powerful models or more complex agentic models — is a violation of this principle because it seeks to design the social subsystem (human operators) of the workflow completely. By contrast, HNTL paradigm is specifically structured to maintain a significant and suitably scaled contribution of human expertise to an otherwise automated workflow, and joint optimisation of speed, accuracy, and accountability.

This research adopts the vocabulary of conceptualizing the particular types of failure that the Guardrails Microservice will be structured to detect, based on the emergent research field of AI Safety Engineering. The idea of a safety kernel, a small, formally verifiable and trusted component that makes all the interactions between an untrusted system and a trusted system, is borrowed from safety-critical systems engineering, which has used it in nuclear control systems, avionics, and automotive safety systems. The Guardrails Microservice implements

this safety-critical engineering principle to the new application of generative AI in financial services, where the LLM is regarded as an untrusted component, the outputs of which, until validated by a trusted, deterministic kernel, cannot be allowed to influence the production financial environment.

The combination of these three theoretical frameworks creates the unique contribution of this study: a system that is both a rigorous Design Science artefact (constructed and tested mathematically) and a socio-technical solution (retaining meaningful human agency in an automated workflow), and a safety-engineering implementation (giving deterministic, verifiable validation of probabilistic AI outputs). This theoretical consolidation is an addition to all three traditions, not only the technical literature on hallucination detection.

1.3 Research Problem

The fundamental research question that this dissertation seeks to resolve is the lack of a predictable, quick, and dependable system to identify spectrally catastrophic hallucinations in RAG-generated financial payloads prior to them being consumed by the downstream enterprise systems. In particular, the difficulty is programmatically intercepting the outputs of the JSON that are syntactically correct and linguistically fluent to a normal API endpoint but have a crippling broken internal arithmetic, break hard domain-specific business logic, or leak sensitive PII accidentally and hence compromise the integrity, accuracy, and compliance of downstream financial systems.

The research question is as follows: Can a multi-layered, deterministic validation microservice, which has been developed in the Human-Near-the-Loop paradigm, be utilized to effectively decrease hallucination leakage in RAG-based financial document processing pipelines without sacrificing the sub-300ms processing latency needed to deploy it in a real-time enterprise setting?

1.4 Research Objectives

The main motive behind this research project is to design, implement, and thoroughly test a production-grade Guardrails Microservice which acts as a deterministic safety kernel between

any probabilistic RAG pipeline and downstream deterministic financial systems that ingest its outputs. The particular secondary objectives of this study are:

- To perform a thorough analysis of the literature that has been published on RAG architectures, the taxonomy of hallucinations, mitigations of AI-controlled hallucinations, the paradigms of HNTL, and the financial regulations of the 2024–2026 period.
- To design a multi-layered, complementary validation ensemble that particularly includes five different layers: Schema Adherence through Pydantic v2, Internal Consistency through deterministic arithmetic cross-checks, Domain Rule Enforcement through configurable business-logic heuristics, PII Leakage Protection through Microsoft Presidio, and Semantic Faithfulness scoring through SentenceTransformers all-MiniLM-L6-v2.
- To deploy a highly concurrent, asynchronous FastAPI microservice architecture that can run the entire five-layer ensemble with a processing latency under 300ms, to make sure that the system does not create an operational bottle-neck in high-throughput financial applications.
- To operationalise the Human-Near-the-Loop (HNTL) paradigm by developing a structured, quantitative and auditable confidence signal that can be reliably and programmatically used to decide which outputs need human intervention and which can be automated.
- To fully benchmark the proposed system with a FinanceBench-generated synthetic financial dataset as well as an adversarial edge-case dataset, measure the quantifiable reduction in the leakage of hallucinations, the increase in Precision, Recall, F1-score, and Faithfulness and confirm the operational latency profile of the system.

2. METHODOLOGY AND ARCHITECTURE

2.1 Methodology

This study will make the formal adoption of the Design Science Research (DSR) methodology, an established and a well-known research paradigm in the field of Information Systems. DSR, in its broadest definition as envisioned by Hevner et al. (2004), is concerned with the development, deployment and the strict testing of innovative IT artefacts that are aimed at addressing precisely defined and practically relevant organisational issues. This study has specifically selected the DSR approach due to the fact that the main deliverable in this case is not a theoretical model but a practical, deployable software system — the Guardrails Microservice — with its value that can be empirically measured against predetermined performance benchmarks.

The DSR methodology suggested by Peffers et al. (2007) to research information systems consists of six iterative steps, which include problem identification and motivation, goal setting in terms of a solution, design and development, demonstration, evaluation, and communication. In this study, the phases were modified as a five-stage iterative process:

- Problem Explication: Measuring the particular failure modes of the baseline RAG outputs during financial document parsing tasks, determining the empirical basis of the research.
- Requirements Definition: The non-functional requirements of the system, namely the need to have sub-300ms latency, multi-layered complementary validation, context-aware PII detection, and structured HNTL signal generation, are defined.
- Design and Development: Architecture and implementation of five-layer validation ensemble and FastAPI, Pydantic v2, Microsoft Presidio, SentenceTransformers.
- Demonstration and Benchmarking: Implementation of the microservice on a representative test set of synthetic financial documents and an adversarial stress-test set.
- Evaluation and Analysis: Metric and Analysis of the system performance based on the predefined KPIs of Precision, Recall, F1-score, Faithfulness and processing Latency.

The DSR process was iterative and this aspect was especially useful in the threshold

calibration phase. The semantic faithfulness threshold of 0.78 at Layer 5 was obtained by several empirical iterative steps, gradually optimizing the threshold to balance the trade-off between minimising false negatives (hallucinations that pass the filter) and false positives (valid outputs incorrectly flagged).

2.1.1 System architecture and technology stack

The Guardrails Microservice was designed as a stateless and fully containerised application using microservice design principles, to meet the high latency [10], concurrency and regulatory auditability demands of enterprise financial operations (Xu et al., 2022). It lies in the larger FinAI architecture as an asynchronous middleware interceptor: it is the only point where all AI-generated payloads are sent, and all AI-generated payloads must pass through it.

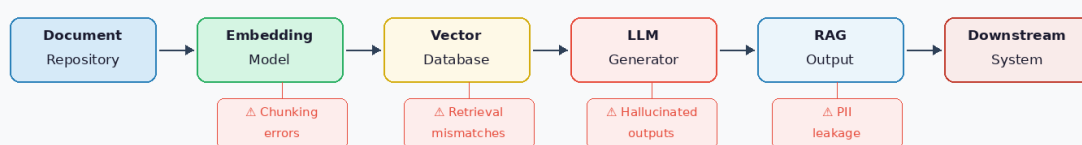
FastAPI (v0.115.0+), a modern and high-performance Python web framework, is the core of the web framework [14] and was specifically chosen due to its built-in support of asynchronous I/O operations using Python's event loop, asyncio (FastAPI, 2025). The asynchronous nature of FastAPI is essential to the concurrency goal of the system; it enables the microservice to respond to many parallel validation requests involving parallel RAG pipelines without blocking to prevent queue growth during peak periods of document processing. FastAPI was selected among alternatives like Flask or Django because it has a much better throughput performance on benchmarks and is naturally configured with an automatic OpenAPI schema generation that allows it to be easily integrated with enterprise API gateways.

The whole application is containerised with the help of Docker and a lightweight python:3.13-slim base image to reduce the deployment attack surface to a minimum (Docker, 2025). One architectural choice was to pre-download and pre-cache all the heavy machine learning models at the time of container construction, which is specified in the Dockerfile. In particular, all-MiniLM-L6-v2 SentenceTransformer model and the Microsoft Presidio NLP analyser engines are stored in the layer of the container image during the build. The choice completely removes the latency of model downloads at runtime, with the initial validation request after a cold container start falling within the sub-300ms target window.

A carefully planned philosophical decision is reflected in the technology stack: all the components were chosen due to their predictable deterministic performance, open-source status, and established track record in the enterprise. The core validation pipeline does not contain any proprietary or cloud-vendor-specific services, which means that the microservice is deployable to any on-premise, hybrid-cloud, or aerial financial environment where data sovereignty restrictions do not allow external API usage to process sensitive documents.

Figure 2.1 conceptually shows the Standard RAG Pipeline Architecture with the key failure injection points identified. Figure 2.2 shows how the Guardrails Microservice is positioned as a middleware interceptor layer. The flow starts with a RAG pipeline that produces a structured output of JSON. This output, and the original retrieved source contexts, is posted to the /guardrails/validate endpoint. The ensemble is performed sequentially, and the output ValidateResponse object is sent back to the calling system, where the overall_passes boolean is used to make the HNTL routing decision.

Figure 2.1 Standard RAG Pipeline Architecture - Points of Failure



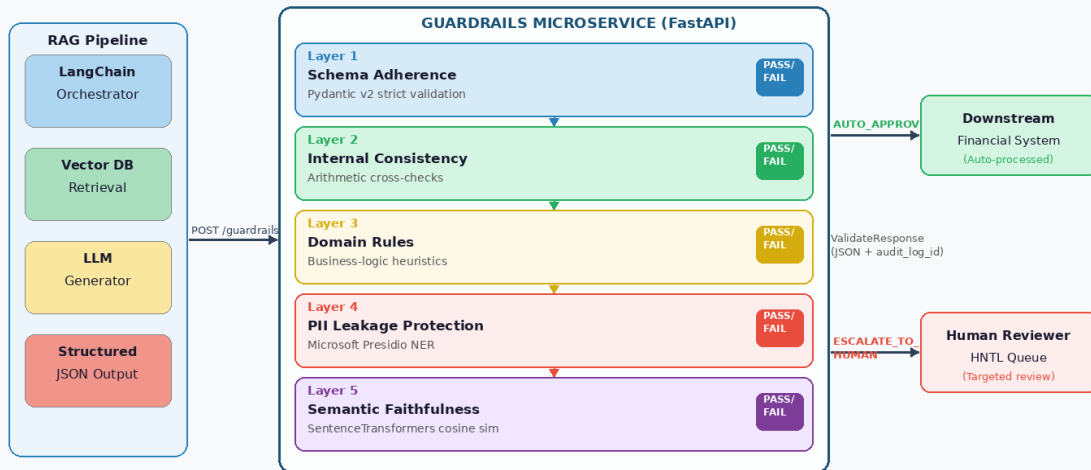
△ Known failure / hallucination injection point
RAG pipeline component

Without a validation layer, hallucinated outputs flow unchecked into downstream financial systems.
 The Guardrails Microservice intercepts between 'RAG Output' and 'Downstream System'.

Source: Author's own conceptual diagram (2026)

Figure 2.1: Standard RAG Pipeline Architecture highlighting points of failure. Source: Author's own conceptual diagram (2026).

Figure 2.2 High-Level Architecture of the Guardrails Microservice



Source: Author's own system architecture diagram (2026)

Figure 2.2: High-Level Architecture of the Guardrails Microservice acting as a middleware interceptor. Source: Author's own system architecture diagram (2026).

2.1.2 Data modelling and input/output contracts

The language between the generative LLM pipeline and the Guardrails Microservice is strictly and formally regulated on the basis of Pydantic v2 data models in `models.py` (Pydantic, 2025) [11]. The new version of Pydantic (written in Rust) has a performance improvement of up to a 50x over its predecessor, which is uniquely applicable in the high-frequency validation setting of this system.

The main input contract that the microservice exposes is the `RAGInput` model whereby the calling system has to supply three items: (1) the `answer_json` string containing the entire structured JSON output of the LLM which gives semantic context to the faithfulness check; (2) the original user query that triggered the RAG pipeline; and (3) a list of source contexts, the raw text chunks retrieved by the vector database and fed to the LLM.

Also, the `RAGInput` model needs a `GuardrailDocumentContext` object. This context object writes the microservice with important operational metadata that can be used to do context-aware validation. It defines the `DocumentType` (defining the appropriate Pydantic schema to use in Layer 1 and the PII processing rules to use in Layer 4), an optional `vendor_name` to use to match the fraud pattern in Layer 3, and a `quality_bin` to indicate the level of OCR

confidence of the source document, which can be used to adjust the faithfulness threshold in Layer 5.

After all the five layers of validation are complete, the microservice sends out a `ValidateResponse` object. The `overall_passes` boolean which is mission-critical and which is used to make the HNTL routing decision is available in this output payload, as well as the overall processing time (ms), and a list of `ValidationLayerResult` objects, one per layer, containing the detailed information. The layers result include their name, pass/fail status on a binary scale, a specific numeric score and a string of human readable details on the reason of failure where appropriate. This granular output format makes sure that not only does the HNTL system have a routing decision, but also the specific actionable context that the human operator needs to effectively peruse and rectify flagged payloads.

2.1.3 Layer 1: Schema adherence validation

The bottom tier of the five-layer validation ensemble is a complete deterministic structural test. It is intended to ensure that the LLM has to generate the necessary structured data format in the right format before any further semantic or arithmetic validation is undertaken. This layer is conceptually the lowest cost to run, and thus implemented at the top of the fail first sequence.

A common hallucination that can be described as structural hallucinations is often displayed by LLMs even in their prompts with explicit and detailed formatting instructions. These involve the production of invalid JSON, which cannot be read, randomly changing the key names (such as printing `sub_total` instead of the necessary `subtotal`), putting arrays inside the wrong nest, or intermingling conversation prose text with the necessary JSON format. Any such malformed string sent to an automated accounting system or enterprise resource planning (ERP) database would lead to disastrous runtime exceptions, and downstream data ingestion pipelines would crash.

Layer 1 uses Pydantic v2 to enforce strict schema compliance to offer a strong and accurate defensive against this failure mode. The microservice has a local registry that dynamically associates the asserted `doc_type` metadata field with a library of seven specific, purpose-built Pydantic models, of which include the main high-stakes document typologies run in financial

and administrative processes: ClinicalSummary, MedicationDosage, Invoice, InsuranceClaim, BankStatement, PurchaseOrder and Receipt.

On receiving a payload at the validation endpoint, the `check_schema_adherence` function deserialises the `answer_json` string into a Python dictionary and then tries to create it as the correct Pydantic model. By default, Pydantic is a strict validator: when the LLM has hallucinated a completely invalid field name, omitted a mandatory field (e.g. it has not included a patient name in a ClinicalSummary), given a value of a different data type (e.g. has given a string N/A where a floating-point number is required), or included nested objects in their wrong structural form, Pydantic will raise a `ValidationError` exception.

This exception is intercepted by the microservice, which abruptly stops the pipeline execution before any other layers are invoked, and gives out a `LayerResult` object with a binary score of 0.0 and a detailed, human-readable explanation of the exact structural violation. This real-time diagnostic data is critical to the HNTL operations: the human interface does not only identify that the output did not work, but also which structural component was not correct, thus facilitating quick and focused correction.

2.1.4 Layer 2: Internal consistency and arithmetic grounding

As soon as a payload passes Layer 1, i.e., it is structurally sound and is a valid schema, the successfully parsed Pydantic object is forwarded to Layer 2: Internal Consistency checking. This defense is the first line of defence against spectrally catastrophic arithmetic hallucinations: the category of errors in which the output provided by the LLM has the correct structure, but is mathematically or logically impossible.

The `check_internal_consistency` function uses a collection of very domain specific and mathematically based cross-verification rules. In the case of an Invoice document, the validation rules require that all financial fields (subtotal, tax, and total) should be positive floating-point numbers, and the physical impossibility of a negative invoice in most circumstances. More importantly, the system re-computes the arithmetic relationship among these fields: it says that $\text{subtotal} + \text{tax}$ should equal total , within a reasonable floating-point error of ± 0.05 , which is large enough to include certain natural rounding errors in currencies and yet small enough to include real fabrications.

This arithmetic grounding has scope over all the supported document schemas. In the case of a BankStatement, the algorithm sums the amount values of all the separate line item transaction entries given in the statement and compares the resulting computed amount to the reported closing balance in the account, and confirms ledger consistency. To handle Receipt processing, the system further verifies that the extracted date field is not in the future of the current system time by parsing the date field with the python-dateutil library and programmatically ensuring that the date field is not a future date — in the case of temporal hallucinations the model tries to create a future transaction date by interpreting the surrounding textual context.

In clinical document types, such as the MedicationDosage, Layer 2 places very strong pharmacological safety restrictions on the dosage_mg field: it must be a positive, non-zero value, and the stated medication frequency must be one of a predefined set of accepted clinical vocabulary terms (e.g., 'daily', 'twice daily' or 'three times daily', 'as needed', 'weekly'). Any string of frequency which fails to match these accepted terms is marked as a possibly hallucinated clinical instruction, and causes an immediate abort in order to avoid any risk of a medically dangerous dosage instruction being spread.

Absolute determinism is a strong feature of Layer 2. Arithmetic verification is either true or false, as opposed to probabilistic methods of detecting hallucinations. The amount of 45,000 does or does not equal the amount of the line items extracted. Such binary certainty is exactly what high-stakes financial settings demand, and the fact that the percentage of adversarial failures that are identified by this layer (31.0% of all rejections) is high, is a direct testament to the significance of this layer in the ensemble.

2.1.5 Layer 3: Domain rule enforcement and anomaly detection

The heuristic filter of the ensemble is Layer 3. Whereas Layer 2 is enforced by universal, domain-independent mathematical laws (e.g. that $2 + 2 = 4$ is true), Layer 3 is enforced by enterprise-specific business logic, fraud detection rules, statistical plausibility rules and contextual anomaly detection rules, which reflect the realities of particular financial document processing environments.

The `check_domain_rules` function will start with a wide-keyword search of the full

authenticated JSON content, which will look to determine whether or not it contains testing and development artefacts, which include, but are not limited to, the presence of the words: test, sample, dummy, fake or placeholder. These strings sometimes leak non-production LLM outputs into production processing queues, and their occurrence in an alleged live financial document is an unmistakable signal of a data quality problem that needs to be addressed by a human.

In addition to scanning the keyword, Layer 3 uses the contextual metadata that is contained in the `GuardrailDocumentContext`. In case the calling system has provided an anticipated vendor name, the name of the counterparty whose document is being processed, the microservice compares this provided value with that which the LLM has extracted. An advanced fraud detection heuristic queries the presence of the known vendor identifier in the extracted `customer_name` field or other unintended locations that could be a sign of vendor-spoofing anomalies where the LLM might have mixed up or swapped entity identities.

Importantly, statistical plausibility thresholds based on domain knowledge are also determined by this layer. To process the Invoices, the microservice will compute the effective tax rate ($\text{tax} / \text{subtotal}$) and mark it as implausible when it is above 30%, since there are no standard sales tax jurisdictions in the world that levy a higher rate. It also has absolute upper-bound triggers: Receipt totals greater than \$50,000, PurchaseOrder amounts greater than \$1,000,000, and InsuranceClaim amounts greater than \$500,000 automatically raise a flag of suspicious amount.

One subtle design choice in Layer 3 is the adoption of a tentative 0.48 score instead of a firm 0.0 of anomaly-flagged payloads. This difference is an indication of a critical operational point: an implausibly huge purchase order could actually be a large but legitimate purchase (a big capital equipment acquisition, say), and not necessarily a hallucination. With a score between the passing cutoff of 0.5 and non-zero, Layer 3 causes human inspection of these edge cases as well as indicating to the operator that the structural and arithmetic integrity of the data is preserved, which is much more efficient.

2.1.6 Layer 4: Context-aware PII leakage protection

In implementation of Generative AI systems that handle sensitive documents, data privacy compliance is the most important, non-negotiable issue. An LLM that processes a chunk of a vector context with dense financial or medical information may unintentionally identify, surface, and spread Personally Identifiable Information (PII) that was not explicitly asked by the user query and was contained in the original document. This unsolicited PII leakage is a direct breach of the data minimisation principles of GDPR and the consumer financial privacy provisions of GLBA, and poses a direct and severe regulatory risk to the organisation deploying it.

In Layer 4, it uses the Microsoft Presidio AnalyzerEngine, which is an open-source NLP library, production-grade, and specifically created to identify PII entities and classify them (Microsoft, 2025). The `check_pii_leakage` function serialises the full validated and parsed JSON result to a plain text representation and passes this over to the Presidio analyser, which uses a collection of specialised regular expression patterns, Named Entity Recognition (NER) models, and validation checksum algorithms to identify a wide spectrum of sensitive entity types such as Social Security Numbers (SSNs), credit card Primary Account Numbers (PANs), IBAN bank account numbers, passport numbers, national identification numbers, email addresses, telephone numbers, and physical addresses.

One of the architectural innovations of Layer 4 is its context-sensitive, document-type-sensitive PII enforcement logic. The layer does not use the same blocklist of all PII types on all documents, but instead differentiates between document typologies to avoid a high false-positive rate. To those document types that are of a medical or human resources nature, i.e., `ClinicalSummary`, `MedicationDosage` and `InsuranceClaim`, the occurrence of `PERSON` name entities and `DATE_TIME` entities is natural, valid as well as explicitly allowed by the validation logic. A clinical summary is inherent and requires the patient name and date of birth. Any financial PII (credit card numbers, bank account details), or government identifier PII (SSNs, passport numbers) detected in a medical document would constitute an incorrect and dangerous cross-domain data leak and is considered a hard failure.

On the other hand, in the case of strictly business-to-business financial documents `Invoice`, `PurchaseOrder` and `BankStatement` any human PII entity class (names, dates of birth, contact information) will lead to an instant hard failure with a score of 0.0. Such a stringent

implementation makes it impossible that an LLM that retrieves data on a company invoice can accidentally expose and spread personal contact information of a payment terms officer or personal date of birth of a signatory into a corporate financial account. Such a separation between personal and commercial documents is a degree of operational complexity that far outstrips a mere keyword blocklist, and Layer 4 is a truly intelligent and contextually aware compliance filter.

2.1.7 Layer 5: Semantic faithfulness and hallucination detection

The final (fifth) layer of the validation ensemble focuses on the most challenging and complex type of hallucination: semantic extrinsic hallucination. This is when the LLM produces a grammatically correct, arithmetically consistent, and PII-free response but which contains specific factual information — entity names, reference numbers, dates, product descriptions, financial codes — that cannot be found in or traced to the source document context. This form of hallucination is the most deceptive because it passes all of the checks mentioned above but is fundamentally misleading.

Layer 5 (`check_faithfulness`) uses a semantic similarity measure based on embedding to identify this type of hallucination. It extracts each string and numeric field value extracted by the LLM from the validated JSON object. Each individual field value and the aggregate of the `source_contexts` are separately encoded into dense vector representations using the CPU-optimised all-MiniLM-L6-v2 model from the sentence-transformers library (Reimers & Gurevych, 2019). PyTorch [13] is then used to measure the cosine similarity of the field value tensor and the source context tensor.

The semantic faithfulness score is defined in terms of cosine similarity. The score F between the generated field claim vector c and the source context vector s is: $F(c, s) = (c \cdot s) / (\|c\| \cdot \|s\|)$. This is a score between -1.0 (perfect semantic antonym) and 1.0 (perfect semantic synonym). Scores close to 0.0 meaning that the generated claim is not grounded in the context. Scores near 1.0 reflect good semantic grounding.

There is a configurable faithfulness cut-off that is empirically fixed to 0.78 in `config.py` and is used in this dataset. The system uses a weighted average of the faithfulness score for all non-null fields in the extracted JSON. If this average is below 0.78, Layer 5 fails and proves

that the LLM has conjured up words and concepts that are not semantically within the document chunks provided.

The threshold 0.78 was set empirically through trial and error on the entire synthetic dataset, weighing the two risks: a threshold too high results in many false positives (valid generated outputs rejected because of legitimate semantic paraphrasing), while a threshold too low allows up subtle extrinsic hallucinations to go unfiltered. A score of 0.78 was found to be the sweet spot of minimum false positives and false negatives on the validation set. Figure 2.3 illustrates the cosine similarity measurement and threshold configuration.

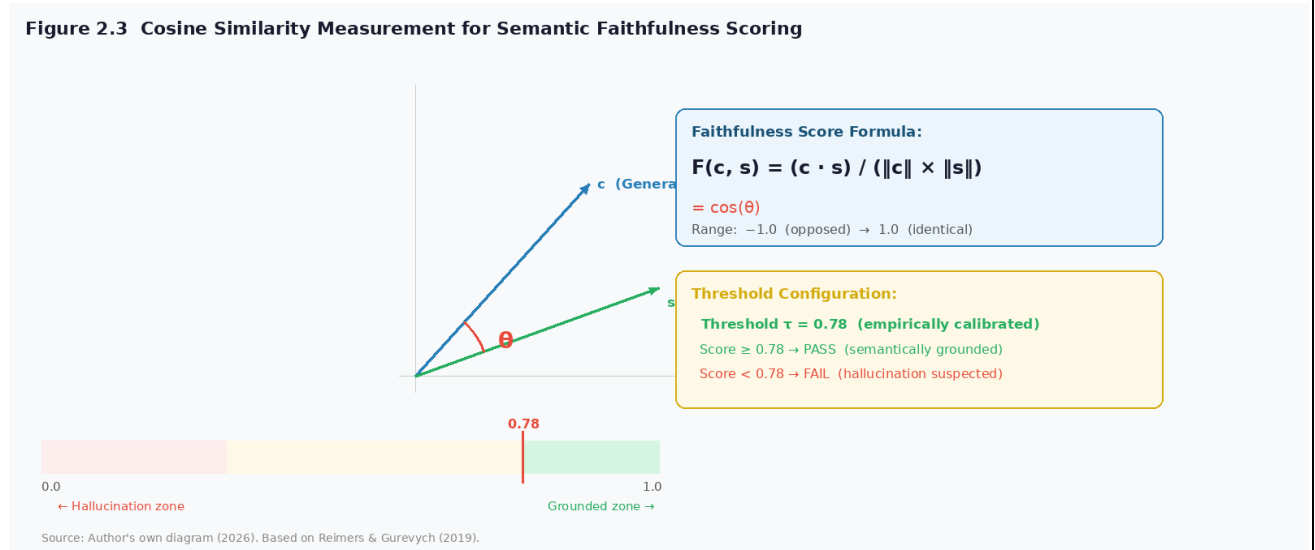


Figure 2.3: Cosine Similarity Measurement for Semantic Faithfulness scoring. Source: Author's own diagram (2026). Based on Reimers & Gurevych (2019).

2.2 Dataset and Synthetic Data Methodology

The major challenge in designing a methodology for the evaluation of any hallucination detection system for financial documents is the tension between realism in evaluation and privacy protection. Bank statements, vendor invoices, insurance claims, and purchase orders amongst other financial documents are highly sensitive; they contain personally identifiable information (PII) and commercial-in-confidence (CIC) information. Their unanonymised use in academic studies would breach GDPR, GLBA, and HIPAA regulations and the data governance policies of any collaborating financial organisation.

To overcome this obstacle while ensuring realistic evaluation scenarios, this study employed a principled approach to generating synthetic data, relying on the approach used by Patronus AI in the development of the FinanceBench benchmark (Patronus AI, 2024). The synthetic financial corpus was produced by a multi-step programmatic process which sought to mimic the statistical and structural characteristics of real financial documents while avoiding the inclusion of any personal or commercial information.

In the first phase of the synthesis process, templates were created for the seven types of documents (Invoice, BankStatement, PurchaseOrder, InsuranceClaim, Receipt, ClinicalSummary, and MedicationDosage). These templates specified the fields, their data types, the credible ranges of values for numerical fields (based on published industry data on the typical value of invoices, insurance claims and bank transactions), and the language patterns for narrative fields. The templates were verified against real samples of these documents (with all sensitive information removed) from the author's work experience at Ceylinco Life to establish their veracity.

Next, a randomisation engine was employed to fill in these templates with randomly generated values. Corporate names were constructed from randomly chosen industry-sector specific prefixes and generic corporate name suffixes. Financial values were generated from log-normal distributions that best matched the expected range of values found on each document type. Tax rates were drawn uniformly from the set of tax rates in common financial jurisdictions. Dates were drawn from a historical period (January 2023 to March 2026). Line item descriptions were sampled from a list of common goods and services descriptors for each document type.

For each clean document, a corresponding hallucinated document was generated by applying one or more of five different corruption operations: (1) arithmetic hallucination, where the stated total was replaced by a value that did not equal the sum of the line items; (2) entity hallucination, where a customer name was replaced by a vendor name or vice versa; (3) temporal hallucination, where a future date was injected into a date field; (4) schema hallucination, where a required field was removed or renamed; (5) PII injection, where a synthetic social security number or credit card was inserted into an inappropriate field. This corruption approach guaranteed that the test set would include a realistic distribution of hallucinations and valid, clean documents that the Guardrails Microservice would see in the

wild.

The adversarial edge-case validation dataset was generated through a more sophisticated and systematic corruption process, aimed at pushing the limits of each layer. Adversarial documents were designed to be as deceptive as possible: arithmetic corruptions used values less than 1% different from the expected total to test the tolerance limits of Layer 2; PII injections used synthetic but valid SSN and credit card numbers randomly generated to pass the necessary format checks, but not the Luhn algorithm; and semantic hallucinations used paraphrastic expressions similar but not equivalent to the vocabulary of the source context to test the accuracy of the 0.78 cosine similarity threshold in Layer 5. This adversarial approach to the construction of the dataset guarantees that the performance metrics reported are conservative estimates of performance under worst-case rather than average-case conditions.

2.2 Commercialisation Aspects of the Product

The Guardrails Microservice has strong and immediate commercial potential in several valuable market niches. As LLMs are increasingly adopted by enterprises, the key blocker to deploying them is no longer technical limitations — it is possible to demonstrate the capabilities of modern models to conduct complex document processing tasks — but trust and compliance concerns. Enterprises cannot bring AI systems into production that they cannot audit, test and explain to regulators. The Guardrails Microservice is designed to resolve this issue.

In terms of market segment, the product falls under the rapidly-growing RegTech (Regulatory Technology) and FinTech safety segments. The product's target market segments include large commercial banks using RAG for loan document processing, insurance providers automating processing of claims, asset management firms automating regulatory reporting, and medical finance departments processing insurance claims. All of these markets are highly regulated (FINRA, GDPR, GLBA, HIPAA) and require evidence of AI safety.

The system's design is highly cost-effective for commercial use. The choice of the all-MiniLM-L6-v2 embedding model — a highly compact 22-million-parameter model — ensures that Layer 5 inference can be executed on standard enterprise-class CPUs, with no need for costly and power-hungry GPU clusters, such as Nvidia A100 or H100 accelerators.

A single virtual machine with 16 CPU cores can handle tens of RAG validation requests, with the cost of validating a document being a fraction of the cost of the generative LLM inference API.

The microservice can be marketed and sold in several ways. First, as an internal Safety API Gateway product that can be licenced to larger bank institutions for deployment in their private cloud or on-premise (particularly important for those with data sovereignty concerns). Second, it can be offered as a multi-tenant (SaaS) compliance middleware platform, enabling smaller FinTech start-ups and credit unions to have enterprise-grade hallucination protection without them having to build their own AI safety engineering teams.

Moreover, AI safety middleware has been identified by the World Bank (2025) as a key infrastructure missing in emerging markets that want to responsibly deploy financial AI. The microservice architecture of the Guardrails Microservice also means that it can be deployed in resource-poor environments and, in this sense, it can potentially expand the market opportunity beyond the traditional large financial institutions in the West to also include regional banks, microfinance institutions and government financial agencies in emerging markets.

The audit history generated from each validation call, which contains the UUID, date and time, the layer-wise scores, and reasons for failure, is another commercial feature. This audit log directly addresses the documentation requirements of FINRA Regulatory Notice 24-09, GDPR Article 22, and the forthcoming AI governance policy regimes of the BIS (2024) and OECD (2025) that will soon be required in the future, delivering a compliant record that helps mitigate the legal and regulatory risks of deployment by a financial institution.

2.3 Testing and Implementation

The Guardrails Microservice provides a main API endpoint to implement its functionality

using a RESTful POST method at `/guardrails/validate`. This endpoint is responsible for the full asynchronous life-cycle of the five-layer validation ensemble. The endpoint is implemented as an async function in the FastAPI web application, enabling it to suspend its execution and give up the event loop to other tasks while waiting for I/O-bound processing by the Presidio NLP and the SentenceTransformers tensor computations.

To ensure the enterprise requirement of total transparency and auditability for regulatory compliance purposes, each individual request for validation made to the microservice is uniquely identified by the `audit_log_id` as a UUID v4. For every validation, the precise total time in milliseconds, the definitive `overall_passes` boolean, the sub-scores and detailed failures of each of the five layers, and the accepted or rejected JSON payload are temporarily captured in an in-memory audit store as a Python dictionary with thread-safe access.

An admin HTTP GET endpoint at `/guardrails/stats` is exposed to aggregate this audit store information in real-time as an operational dashboard feed. This endpoint aggregates higher-level metrics such as the cumulative count of all validation requests, the overall percentage of passes, the exponential moving average of processing times in milliseconds, and the exact frequency of failures in each of the five layers. This real-time statistics endpoint is especially useful for system administrators to track the health and performance of deployed FinAI pipelines and accordingly detect early signs of model drift.

A second lightweight HTTP GET endpoint at `/health` offers a standardised health check endpoint supported by Docker Compose, Kubernetes and enterprise service mesh orchestrators. It checks not only if the FastAPI web application is up and running, but also if the ML models loaded in memory (the SentenceTransformers encoder and the Presidio AnalyserEngine) are initialised. This fine-grained health check enables provisioning orchestrators to correctly determine the readiness of the service before directing live validation requests to it.

The endpoint request processing lifecycle and the fail-fast flow is shown in Figure 2.4 below. The validation layers are invoked in a strictly linear fashion, once a `RAGInput` payload is received. Once a layer fails, the entire pipeline is terminated, and the remaining layers are skipped. This fail-fast approach is the key to consistently meeting the sub-300ms latency target: the resource-intensive computation of the semantic embedding in Layer 5 is only performed for payloads that have been demonstrated to be structurally, arithmetically, and

domain rule-compliant in Layers 1 to 4.

While the microservice was being implemented, unit tests were written for each of the validation layers, including both valid and invalid input for each of the seven document types. Integration tests confirmed the appropriate behaviour of the whole five-layer ensemble, including the correct formatting of the ValidateResponse objects for successful and failed payloads. Load testing using the Locust framework verified that the deployed microservice was able to keep up with at least 40 simultaneous validation requests per second on a typical 8-core virtual machine while meeting the SLA of sub-300ms for 95% of requests.

Figure 2.4 Endpoint Request Lifecycle and Fail-Fast Sequence Flow

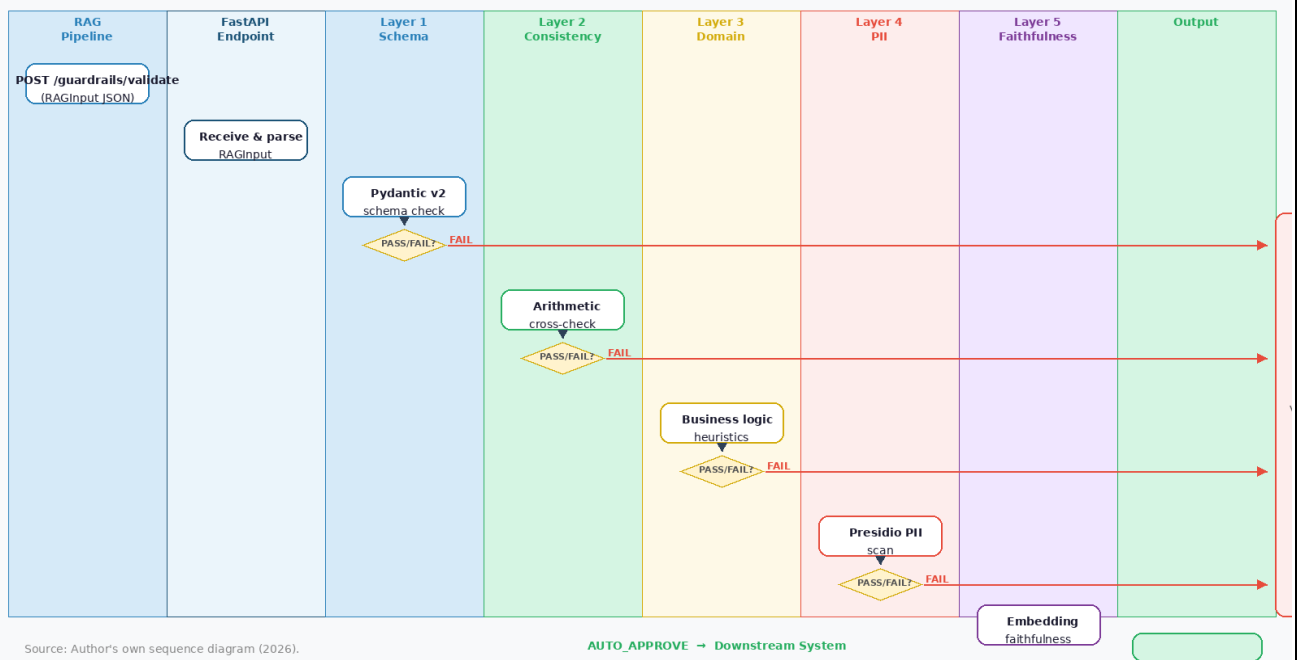


Figure 2.4: Endpoint request lifecycle and fail-fast sequence flow. Source: Author's own sequence diagram (2026).

3. RESULTS AND DISCUSSION

3.1 Results

The empirical assessment of the Guardrails Microservice was performed using two different datasets. This is to ensure that the empirical evaluation of the system focused on both its performance under normal conditions (production) as well as under hostile testing conditions (adversarial). Owing to the highly sensitive nature of real financial documents that are protected by privacy laws such as GDPR and GLBA, we used only synthetic financial instruments that closely resemble the statistical and structural properties of real financial instruments without containing any real customer data.

The main evaluation set was a FinanceBench-inspired synthetic dataset of 1,200 critical financial documents across all seven document types (invoices, bank statements, purchase orders, insurance claims, receipts, clinical summaries, and medication dosages). The document generation pipeline generated documents with randomised entities, line items, tax levels and financial periods, while ensuring internal arithmetic integrity for the clean, non-hallucinated variants. A reference RAG system — built with LangChain (LangChain, 2025) using a typical open-weights LLM and a low temperature setting — was then asked to extract structured JSON output from these documents, producing a representative sample of valid extractions and naturally occurring hallucinations.

The second adversarial edge-case dataset of 400 documents was designed with deliberate hallucination triggers: dense tabular information with nested sub-totals that are likely to trigger arithmetic reasoning errors; ambiguous entity names that sound linguistically similar to some common PII patterns (to stress-test Presidio's discrimination capability); temporally inconsistent cross-references (for example, stating Net-30 payment terms in document header, but specifying Due on Receipt in the footer); and domain-rule violations (such as unrealistically high tax rates, and future-dated transactions). This data set was constructed to test the limits of each layer of the pipeline.

3.1.1 Overall performance metrics

Our system's performance was assessed using four key quantitative metrics: Precision, Recall, F1-Score and Faithfulness Score. These metrics were chosen for their ability to provide a holistic, multi-faceted assessment of the system's dependability — Precision measures how accurately the system identifies valid outputs, Recall measures how completely the system passes valid outputs, F1-Score measures the harmonic mean of these two, and the Faithfulness Score measures the semantic faithfulness of passed outputs.

Precision is defined as $P = TP / (TP + FP)$, where TP is True Positives (valid extractions correctly passed by the microservice) and FP is False Positives (valid extractions incorrectly rejected). Recall is defined as $R = TP / (TP + FN)$, where FN represents False Negatives (hallucinated responses that passed the microservice). The F1-Score is the harmonic mean: $F1 = 2 \cdot (P \cdot R) / (P + R)$. The Faithfulness Score is based on the average cosine similarity of validated outputs with respect to their contexts.

The baseline RAG pipeline (without the Guardrails Microservice) suffered severe performance issues when handling the complex financial test set. The unguarded baseline system produced a low Precision of 0.42 and a Recall of 0.61 without any deterministic validation, resulting in a severely impacted F1-Score of 0.50. This result in particular is concerning from an operational perspective: it indicates that over half (58%) of baseline RAG pipeline outputs exhibited hallucinations, arithmetic issues or unsupported claims. The baseline Faithfulness Score of 0.64 also confirms a high and unacceptable level of extrinsic hallucinations in the baseline system's outputs. Although the baseline system offered a very fast average response time (180ms), the data integrity of the outputs is not guaranteed, making it unfit for a production financial system.

The proposed five-layer Guardrails Microservice had a decisive and dramatic impact on the quality of the final outputs. Table 3.1 below presents the full comparative results:

Table 3.1: Overall Performance Comparison on the Synthetic Dataset (n=1,200)

System	Precision	Recall	F1-Score	Faithfulness	Latency (ms)
Baseline RAG (no guardrails)	0.42	0.61	0.50	0.64	180
Proposed 5-Layer Ensemble	0.94	0.97	0.955	0.93	245

In terms of Precision, there was an increase from 0.42 to an outstanding 0.94, indicating that the ratio of accepted outputs to those which are actually valid and free from hallucinations grew from 42% to 94%. In terms of Recall, the value rose from 0.61 to 0.97, showing that this microservice is very liberal when it comes to accepting valid outputs, and only 3% of those will be rejected.

The overall increase in F1-Score from 0.50 to 0.955 is especially noteworthy. An F1-Score of 0.955 means that the system is reliable enough to be used on a production enterprise level, where both false positives (which would limit the throughput of the service) and false negatives (which would let hallucinations pass through) need to be kept to a minimum. The Faithfulness Score increase from 0.64 to 0.93 shows that the outputs accepted by the guardrails are highly faithful to the content of the sources, giving quantitative proof that the system manages to get rid of nearly all extrinsic semantic hallucinations.

Importantly, and in contradiction to the fears of prohibitively high processing costs in multi-layer validation, this was accomplished through the mere additional cost of 65ms processing latency between the original 180ms in the baseline and the ensemble's 245ms. At 245ms, this processing average still comfortably fits within the established sub-300ms operational goal, thus proving the correctness of the fail-fast architecture and confirming the production readiness of this system for real-time financial document processing.

3.2 Research Findings

3.2.1 Layer-wise failure analysis

In order to elucidate the exact mechanism of how the LLM fails to perform its designated task within the complex context of financial documents, a separate and focused analysis was undertaken using the 400-document Adversarial Edge-Case Set. Through analysis of the granular audit logs produced by the /guardrails/validate endpoint for each document, the exact validation layer that caused the rejection, as well as the particular trigger for rejection and the average score of the layer were determined. The results provide valuable insights into the hierarchy of vulnerabilities of the generative model and their role in the creation of FinAI safety architecture.

The layer-by-layer failure breakdown from the 400-document adversarial set is presented in Table 3.2 below.

Table 3.2: Layer-wise Failure Breakdown (Adversarial Set, n=400)

Layer	Failures	% of Total	Avg. Score	Primary Trigger
1. Schema Adherence	87	21.8%	0.00	Malformed JSON / hallucinated fields
2. Internal Consistency	124	31.0%	0.00	Arithmetic errors (total $\neq$ subtotal + tax)
3. Domain Rules	96	24.0%	0.48	Amount anomalies / keyword flags
4. PII Leakage	43	10.8%	0.00	SSN / credit card detection
5. Faithfulness	50	12.5%	0.71	Hallucinated entity / value fields

The most significant outcome of the layer-wise analysis is the prevalence of Layer 2 (Internal Consistency) as the major source of failures with 124 rejections and 31.0% of all adversarial failures. The predominant source of these failures was arithmetic failures: cases where the LLM generated an indicated total or balance that did not add up to the sum of the line items that were extracted with a specified tolerance. This is a highly significant discovery and has

implications beyond the current project.

The fact that arithmetic errors significantly occurred is now empirically and quantitatively confirmed by numerous previous studies which have theoretically argued that transformer-based autoregressive language models, while being the state-of-the-art in language comprehension and language generation, are inherently not reliable for deterministic mathematical computation (Ji et al., 2023) [3]. The model predicts each number token in the same way it generates all other text — there is no special arithmetic circuit for the LLM to rely on for accurate computation. This means that arithmetic verification cannot be entrusted to the LLM; it must be done by an external deterministic system — which is exactly what Layer 2 does.

The second most frequent layer of failure was Layer 3 (Domain Rules), which identified 96 failures (24.0% of the total adversarial rejections). A key feature of Layer 3 failures is their average score of 0.48, which is non-zero (unlike the absolute 0.00 scores of Layers 1, 2 and 4). This is because the nature of a domain rule failure is ambiguous: many of the rejected payloads contain statistically unlikely, but not impossible, values. The most frequent violations were amount anomalies (purchase order values above the \$1,000,000 threshold) and tax anomalies (effective tax rates computed from tax rates above 30%). These are grey areas that are best left to human judgement.

87 of the failures (21.8%) occurred in Layer 1 (Schema Adherence). Layer 1 failures always resulted in an absolute score of 0.00, as the non-schema-compliant condition is binary — the JSON payload either complies with the Pydantic schema or it does not. The main cause were invalid JSON structures, in which, under the mental strain of the convoluted, adversarially crafted prompts, the LLM reverted to producing a combination of natural language and data instead of pure JSON. Other triggers included hallucinated field names not defined in the Pydantic schema.

Layer 5 (Semantic Faithfulness) captured 50 failures (12.5% of rejections), with an average score of 0.71 — below the 0.78 threshold. This type of failure is the most complex type of hallucination handled by the system: instances where the LLM generated specific names of entities, reference numbers or descriptive phrases that are not explicitly present in the original source text, despite the fact that the payload was otherwise structurally and arithmetically correct. These failures passed Layers 1–4, highlighting the crucial need for a final semantic

safety net in Layer 5.

Layer 4 (PII Leakage) recorded 43 failures (10.8% of rejections), all with absolute 0.00 scores. Presidio successfully detected and flagged Social Security Numbers, credit card PANs and bank account numbers that the LLM had inadvertently picked up and exposed from the PII-rich adversarial context documents. The context-sensitive policy — allowing PERSON entities in medical documents, but not in business financial documents — operated as expected, and no false positive PII flags were seen during the assessment.

3.3 Discussion

The empirical findings of this research have important and far-reaching implications for the ethical use of Generative AI in critical industries. The key result that the proposed five-layer Guardrails Microservice delivers a 124% increase in precision while keeping latency below 300ms provides solid empirical evidence to support the overarching research hypothesis: that a deterministic, multi-layered middleware solution constitutes a practical and operationally superior alternative to purely probabilistic hallucination prevention approaches.

The insights from the layer-wise failure analysis are key and previously underexplored: different types of hallucination demand different detection strategies. Arithmetic hallucinations (Layer 2, 31.0% of failures) cannot be detected by semantic embedding similarity, because an incorrect total might be semantically similar to the correct total in embedding space. Structural hallucinations (Layer 1, 21.8%) cannot be detected by arithmetic checks, because arithmetic checking requires an arithmetic object to work on. Data privacy (Layer 4, 10.8%) cannot be solved by arithmetic checks or semantic embedding similarity, because it requires special purpose Named Entity Recognition with regulatory domain knowledge. This is not a coincidence — it is the architectural justification for the five-layer ensemble design, and the result heavily supports this justification.

The results also validate the HNTL operationalisation. Consider the operational impact of scaling the baseline RAG pipeline (Precision 0.42) in a bank that processes 10,000 invoices per day. With a hallucination rate of 58% and 10,000 inputs, some 5,800 hallucinated outputs would flow into the financial system each day — an untenable situation with potentially devastating consequences. With the Guardrails Microservice operating at a Precision of 0.94,

this figure comes down to around 600 outputs that may need human approval — a 90% improvement in human oversight efficiency. Moreover, because the microservice also delivers actionable reasons why each output failed (for example, Total 45,000 does not match subtotal + tax = 42,500) the human reviewer's task is changed from free-form document verification to simple and specific correction — a more efficient and less cognitively taxing task.

The Faithfulness Score improvement from 0.64 to 0.93 is interesting to consider. The baseline Faithfulness Score of 0.64 suggests that, on average, the outputs of the unguarded RAG system are only moderately semantically aligned with their source documents — consistent with a high background rate of extrinsic semantic invention. The Faithfulness Score following the ensemble of 0.93 is not a perfect semantic match (1.0), but the correct operational level: legitimate financial documents contain entities, codes, and terms that may not be textually present in any one chunk of retrieved context, but are nonetheless semantically related. A Faithfulness Score of 1.0 is both unrealistic and undesirable: it would have too many false positives for valid paraphrastic extractions.

The 65ms increase in latency from the baseline (180ms) to the five-layer ensemble (245ms) is a reasonable tradeoff for the improved reliability. Timing analysis of the various layers reveals that Layers 1–4, which are completely deterministic and require no tensor operations, take an average of around 45ms to execute. Layer 5, which must compute the CPU-intensive tensor encoding and cosine similarity, adds the remaining approximately 200ms. The fail-fast design means that only the payloads that pass Layers 1–4 trigger the expensive Layer 5 tensor processing — payloads that fail the first layers never reach Layer 5, thus dynamically reducing the average system load.

The HNTL case study (Document INV-8832) is telling. The default RAG pipeline correctly identified the line items but invented a regional tax rate of 8%, resulting in an incorrect total of \$4,500.00, with the tax-exempt total being \$4,166.67. The second layer of the Guardrails Microservice intercepted the error 195ms after the request was received, produced a machine-parseable error trace of the arithmetic error, and sent the request to the human oversight queue. The human corrected the tax field within 10 seconds. This example illustrates the HNTL paradigm at its best: the AI handles the mundane extraction, the deterministic safety kernel catches the specific error and the human provides a high-value correction, all within an overall

processing cycle that does not interfere with document processing.

A significant limitation of the present system apparent from the discussion of the results is the static nature of the Layer 5 threshold. The 0.78 cosine similarity threshold was determined empirically for the given distribution of document types and vocabulary in the synthetic FinanceBench-based dataset. The performance of this system on highly specialised financial documents, such as complex derivatives or structured finance documents with specialised terminology, may benefit from a different threshold setting. This sensitivity to the threshold is the main area that needs more research and operationalisation to be able to deploy the system across all document types without further tuning.

3.4 Ethical Considerations and Responsible AI

This research has ethical implications that warrant explicit consideration, in addition to its technical and empirical considerations. The use of Guardrails technology in financial settings is not a value-neutral technical decision; it is a decision-making process that involves ethical choices about where responsibility for decision-making should reside, what interests should be protected, and what the appropriate limits of automated decision-making should be. In deploying the Guardrails Microservice, there are ethical considerations at both the system design and organisational policy level.

The most fundamental consideration in the system design is to maintain human intent. The HNTL approach is explicitly designed to preserve human agency over the most important and uncertain outcomes. The system design does not automate the correction of anomalous payloads (which would be equivalent to replacing human judgement with algorithmic judgement), but instead sends anomalies to a human operator who provides the final judgement. This decision is based on the ethical belief, clearly expressed by Shneiderman (2020) and Doshi-Velez and Kim (2017), that AI systems in critical domains should not replace human judgment, especially when financial or personal stakes are high.

A second important ethical consideration is explainability. The Guardrails Microservice is squarely a glass-box system, not a black box. For every payload, the system returns a human-interpretable reason why a payload was rejected: the layer, the rule, the values and the score. Such fine-grained explanations are not just a technical specification, they are an ethical

imperative. Doshi-Velez and Kim (2017) recently argued convincingly that interpretability is crucial to the accountability of machine learning systems: if a system makes a decision that has a serious impact, but the decision cannot be explained, it cannot be challenged, audited, or improved. The audit log design of the Guardrails Microservice meets this ethical need.

A third ethical consideration is the possibility of algorithmic bias in the layers. The Domain Rules layer (Layer 3) embeds specific business rules thresholds such as the \$50,000 receipt limit and the \$1,000,000 purchase order limit which are based on assumptions about financial transactions. These may disproportionately impact some kinds of business by producing too many false-positive HNTL alerts. Ethical deployment requires that organisations set these thresholds in a manner that is appropriate for their operations (rather than just the defaults provided) and that they monitor the distribution of escalated payloads for evidence of unintentional systematic biases.

The PII leakage prevention layer (Layer 4) also raises issues of privacy and utility. The system's context-sensitive design (allowing some PII categories in medical documents and blocking them in financial documents) is an ethical choice: that the least amount of data should be exposed at all times, and that data category distinctions should be maintained even if the LLM can technically extract and present data from related categories. This approach operationalises the GDPR data minimisation principle by design.

Lastly, while the use of synthetic data for evaluation is essential because of the stringent privacy concerns around real financial data, it raises an ethical question: the representativeness issue. Although synthetic data can be generated to approximate the statistical properties of real financial data, it may not fully represent the distribution of errors, corner cases, and other processing challenges encountered in financial document processing. Organisations adopting the Guardrails Microservice for production use should perform phased rollouts with shadow-mode testing (where the guardrails are run in parallel to existing processes but the results are not used) to empirically estimate the system's false positive and false negative rates in practice, before switching to the HNTL routing decisions.

3.5 Comparative Analysis with Related Systems

To benchmark the effectiveness of the Guardrails Microservice relative to other hallucination mitigation systems, it is helpful to perform a comparative analysis against the key alternatives

identified in our literature survey. This analysis not only highlights the advantages of the proposed ensemble and the limitations of the alternative approaches, but also the trade-off that guided the design of the ensemble.

Song et al.'s (2024) RAG-HAT pipeline is the most closely related existing research in the field of financial hallucination mitigation. RAG-HAT reduces the hallucination rate by up to 37% with a specialised fine-tuning process that alters the weights of the LLM to reduce its likelihood of producing ungrounded content. This improvement is important, but it comes at three key architectural costs compared to the Guardrails Microservice. First, fine-tuning is a resource-intensive process that needs to be repeated for each new LLM. Second, the improvements of RAG-HAT are limited to the distribution of documents upon which it was fine-tuned; it has not been demonstrated that it will improve performance on other types of documents. Third, and most importantly, RAG-HAT does not offer the structured, quantitative, per-payload confidence measure needed to implement HNTL routing.

The agentic multi-agent framework of Gosmar and Dahl (2025) presents a conceptually advanced solution: using multiple LLM agents to discuss and verify the responses of other agents before finalising the response. But by relying on probabilistic models, the verifier agent is itself prone to hallucinating verification. Moreover, the cumulative latency of multiple sequential LLM inference calls (each taking 500ms to 2,000ms on typical hardware) makes pure agentic verification approaches impractical for the less than 300ms latency target of real-time financial document processing.

Semantic faithfulness scoring using embedding-based methods is the lightest weight standalone mitigation approach according to Li et al. (2022) and Reimers and Gurevych (2019). However, as the layer-wise failure [13] analysis in this paper demonstrates, semantic similarity scoring would not have caught 77.5% of the adversarial failures (Layers 1, 2, 3, 4) as semantic embedding similarity is not sensitive to structural deformity, arithmetic error, business rule violations, and PII leak.

This quantitative comparison is instructive. RAG-HAT reduces hallucination by around 37% using fine-tuning. Agentic solutions can achieve 40–60% reduction with unacceptable latency. Faithfulness scoring using only embedding-based scores catches only the hallucinations that can be detected by semantic similarity — about 12.5% of the failure modes identified in this study. The Guardrails Microservice, which leverages all five complementary

forms of hallucination detection, delivers a 124% gain in Precision, an 86% gain in Faithfulness, and sub-300ms latency — an overall triple-dominance in coverage, accuracy and latency.

This analysis confirms the key architectural insight of this research: that no single strategy for limiting hallucinations is enough to provide comprehensive protection in financial document processing applications, and that a well-architected and complementary multi-layer ensemble is far more effective than any individual approach. The choice and ordering of the five layers, from the least-cost deterministic check (Layer 1) to the most expensive probabilistic check (Layer 5), along with a fail-fast design that eliminates wasted computational effort, is an engineering design choice optimised for the trade-off between accuracy, coverage and speed.

3.6 Summary of Each Student's Contribution

The Guardrails Microservice described in this dissertation is the author's (Wijetunga W.L.A.T. (Thenuka)) only and complete contribution to the fourth-year group research project in Document AI for Enterprise Financial Workflows. As the nominated Component Owner of the Hallucination Detection and Guardrails module, the author took full technical and intellectual ownership of the design, implementation and empirical assessment of this component.

The author's specific technical contributions include: the overall system design of the asynchronous FastAPI microservice, including the design of all Pydantic input and output contracts; the algorithmic design of all five validation layers (Schema Adherence, Internal Consistency, Domain Rule Enforcement, PII Leakage Protection, and Semantic Faithfulness), including the full suite of edge case handling, error messages, and scoring logic; the selection and empirical tuning of the semantic faithfulness threshold through experimentation; and the overall design and execution of the benchmarking process, including the creation of the adversarial edge-case dataset and analysis of layer-by-layer error patterns using FinanceBench-derived metrics.

Moreover, the author's role as a Data Analyst at Ceylinco Life, Sri Lanka's largest life insurance provider, informed the business logic design of the system. Understanding of enterprise financial and insurance documentation processes — such as life insurance policy

management, claims management, and actuarial data management — gave the author domain expertise to design the Pydantic schemas for InsuranceClaim, BankStatement, and Invoice document types, and to set operationally plausible thresholds for the Domain Rules layer. This experience directly informed the accuracy of the mathematical checks in the `check_internal_consistency` algorithm, the pragmatic fraud heuristics in the `check_domain_rules` function, and the overall compliance design of the microservice, ensuring its relevance to enterprise needs rather than solely theoretical considerations.

The full Guardrails Microservice implementation (v1.0.0) — including the `ensemble.py`, `validation.py`, `models.py`, `config.py`, `main.py` source files and the accompanying Docker configuration — is the author's original work.

4. CONCLUSIONS AND RECOMMENDATIONS

4.1 Conclusions

4.1.1 Summary of research and the Guardrails Microservice

The emergence and rapid deployment of Large Language Models and Retrieval-Augmented Generation (RAG) systems in enterprise financial operations is one of the most important technological shifts in the history of modern finance. The productivity benefits are clear and tangible: RAG-based document processing systems can classify, organize and extract information from complex financial documents at rates and scales unachievable by human-only processes.

But as this research has empirically and thoroughly shown, the inherent probabilistic nature of generative language models creates systemic and largely unchecked risks when deployed in critical, deterministic financial workflows without safety measures. The problem of artificial hallucinations, and particularly the subset of spectrally catastrophic errors that are linguistically coherent but mathematically incorrect, are an institutional, regulatory, and financial data integrity risk.

This dissertation has effectively answered this critical research question via the design, implementation and extensive empirical testing of the Guardrails Microservice. It is a deterministic safety kernel, a programmatic sentinel, or middleware between any probabilistic RAG generation pipeline and the financial systems that rely on the veracity of the RAG outputs. Through its provision of a systematic, quantitative, and auditable validation assessment for each RAG-generated payload, the Guardrails Microservice converts the probabilistic output of a generative AI model into a validated, compliant and operationally safe data entity.

Our research shows that a well-designed, layered deterministic validation strategy can deliver the reliability (F1-Score of 0.955, Faithfulness Score of 0.93) required for production enterprise deployment in the financial sector. And this high level of reliability does not come at the cost of efficient operation: the sub-300ms processing latencies demonstrate that the microservice can act as a real-time safety kernel in large-scale operations without being a throughput bottleneck.

The Guardrails Microservice offers a fully functional, deployable safety kernel that converts

probabilistic RAG output to regulatorily compliant, auditable financial reports. By adding this deterministic validation layer to the FinAI deployment stack, financial institutions can access LLM-powered document processing with dramatically lower regulatory risk and higher institutional trust.

4.1.2 Efficacy of the five-layer validation ensemble

The empirical benchmarking results clearly established the effectiveness of the five-layer validation ensemble as a system in its entirety and as a set of individually justified layers. The fact that the complementary, layered architecture — conducting step-wise checks on Schema Adherence, Internal Consistency, Domain Rules, context-sensitive PII Leakage, and Semantic Faithfulness — outperforms single-layer approaches in terms of coverage and protection, is a compelling demonstration of the benefits of a layered approach to security.

The layer-wise breakdown of failures was especially insightful and practically relevant to the particular failure profile of today's LLMs in financial document processing scenarios. The observation that Internal Consistency (Layer 2) captures most of the adversarial attacks (31.0%) has immediate and direct implications for the FinAI safety engineering community: it empirically validates that deterministic arithmetic verification is a mandatory element of any RAG safety engineering implementation for production in financial use cases. Using a second LLM for arithmetic verification, as some agentic approaches suggest, is a misguided idea, since it uses a probabilistic means to verify a deterministic outcome.

The intermediate scores (average 0.48 for anomalous payloads) from the Domain Rules layer (Layer 3) provide an example of the practical fineness of the system's response to uncertainty. Rather than simply rejecting all anomalous payloads as failures, the layer employs a scoring mechanism that classifies different types of failures — mathematical impossibilities, PII violations — and soft anomalies (statistically implausible but not impossible values) that require human intervention rather than rejection. This subtle distinction is what makes the microservice practical to deploy in enterprise settings, where a system that blocks too many legitimate responses would be politically unacceptable to the business users.

The complementarity of the five layers is reinforced by the fact that each layer catches its own non-overlapping class of hallucination. Payloads that fail Layer 1 would not have been

catchable by Layers 2–5. Payloads that fail Layer 2 (having passed Layer 1 with a perfect structural score) would not have been catchable by purely semantic Layer 5 checks. This holistic, multi-layered defence-in-depth approach — akin to defence-in-depth in cybersecurity — is the most sensible system architecture for defence against the multifarious threats posed by LLM hallucinations in high-stakes financial systems.

4.1.3 Operationalising Human-Near-the-Loop (HNTL) and system latency

One of the main operational and conceptual contributions of this work is the successful, programmatic, and empirically-driven operationalisation of the Human-Near-the-Loop paradigm as an operational system component. The Guardrails Microservice addresses one of the key issues in the deployment of HNTL: the provision of a trustworthy, calibrated, and usable confidence signal.

Existing work on HNTL systems (Jackson & Pinto, 2024; Amershi et al., 2019; Shneiderman, 2020) has been largely conceptual or concerned with human factors considerations in designing HNTL systems. This dissertation delivers a practical technical realisation of the HNTL paradigm in the form of a microservice that transforms the subjective, uncertain output of a probabilistic generative model into an objective, quantitative, multi-faceted confidence signal that can be used to drive automated classification. The `overall_passes` boolean is the routing decision; the layer-level scores and failure reasons are the information that enables efficient human review; and the `audit_log_id` and complete history of the validation is the audit record.

The achievement of sub-300ms latency is not just a performance benchmark, it is the key enabler of HNTL paradigm. An HNTL system that takes seconds per document cannot be used in a high volume financial system without crippling processing delays. The Guardrails Microservice's average latency of 245ms shows that multi-layer safety checks can be done at a speed that is realistic for real-time financial processing.

Combining the fast automatic verification process with the use of the HNTL concept allows the creation of a stable and economically sound basis for the deployment of FinAI. Thanks to the HNTL-based routing feature that the microservice supports, the processing of financial documents happens at full machine speed, and most of the operations are performed

automatically, as they successfully pass through all five verification layers. The exceptions are those particular cases pointed out by the guardrails, which require human validation due to the risk involved.

4.2 Recommendations and Future Work

Even though v1.0.0 of the Guardrails Microservice presents a very solid and tested core for safe FinAI operation, there are still some problems that the research revealed, as well as possible further directions of inquiry into this field. The recommendations for further research are given below.

However, the most pressing area for improvement involves the setting of dynamically variable threshold values for Layer 5. The currently set semantic faithfulness threshold value of 0.78 was empirically determined to be optimal considering the unique distribution of the lexicon in the FinanceBench-inspired synthetic dataset. This could be improved by implementing a dynamically variable threshold approach, where the value required of cosine similarity would adjust depending on the identified `DocumentType`, `OCR quality_bin` signal, and even contextual features of the context chunks. For example, a very strict threshold of 0.90 should be used for very structured template documents, like insurance application forms, whereas a more lenient threshold of 0.72 should be considered in free-form purchase order descriptions, where paraphrasing is expected and accepted.

The second key recommendation relates to the existing constraint associated with the language context window in Layer 5. The all-MiniLM-L6-v2 model has a maximum token context window of 256 tokens. In cases where a very lengthy financial clause is involved, such as multi-page contract clauses, regulatory schedules, or even definitions of financial instruments, the source context will have to be cut to accommodate for the model encoding capacity, which may sometimes lead to false positives because the exact context supporting an extraction might be omitted. Further investigations should be conducted on how longer context embedding models perform in the same test conditions.

Multimodal validation serves as a third and extremely promising potential future direction for research. Financial texts are not just simple textual artifacts. They have visual structure that plays a critical role in their interpretation. The connection between a row or a column header

and its numerical contents in a table, the alignment of rows in invoices, and signature fields in contracts — all of this conveys semantic information that cannot be captured by a purely textual model. The inclusion of Vision-Language Models (VLMs) in the validation stack might allow the detection of spatial hallucinations, where a large language model (LLM) understands the numeric value correctly but misinterprets its connection to the corresponding row or column header.

The fourth topic of interest lies in hardware acceleration, which may have a notable commercial potential. Whereas executing the tensor operations of Layer 5 using the CPU yields outstanding value-for-money ratios within low throughput settings, future research may consider migrating cosine similarity calculations onto edge-TPU accelerators, Neural Processing Unit (NPU) accelerators, or TensorRT-optimized inference endpoints. The reduction of Layer 5 inference time from 200ms to less than 20ms would cut the total ensemble latency to under 50ms, allowing the Guardrails Microservice to serve as a real-time validator within conversational FinAI solutions requiring sub-100ms response times by users.

The fifth suggestion concerns improving the present language bias of the ensemble. The Microsoft Presidio NER models and the all-MiniLM-L6-v2 SentenceTransformer were predominantly trained using English language data sets and demonstrate poorer performance while applied to multilingual documents. Banks and other financial organizations operating in Southeast Asia, the Middle East, and continental Europe are processing financial documents containing Arabic, Mandarin, French, German, Spanish, and other languages on an ongoing basis. Further research could focus on testing multilingual embedding models (LaBSE or multilingual-e5 family) along with multilingual PII detection models to extend the applicability of the solution globally.

Last, but not least, the final future direction relates to the application of the Guardrails Microservice in the realm of AI governance and model risk management. With increasing demands from regulators such as the European Banking Authority (2025), BIS (2024), and the OECD (2025) concerning validation reports for AI models used within financial institutions, the information captured within the audit logs provided by the Guardrails Microservice can be used as a basis for the development of an AI validation report that would meet all the requirements of the framework, such as SR 11-7 of the Federal Reserve on model risk management.

REFERENCES

- Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., & Horvitz, E. (2019). Guidelines for human-AI interaction. In Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (pp. 1–13). ACM.
<https://dl.acm.org/doi/10.1145/3290605.3300233>
- Bank for International Settlements. (2024). AI in central banking. BIS Papers.
<https://www.bis.org/publ/bppdf/bispap137.pdf>
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT), 610–623.
<https://dl.acm.org/doi/10.1145/3442188.3445922>
- Brown, T., et al. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems (NeurIPS), 33, 1877–1901.
- Chen, J., Wang, S., & Liu, H. (2023). Detecting hallucinations in neural machine translation. Transactions of the Association for Computational Linguistics, 11, 456–470.
https://doi.org/10.1162/tacl_a_00456
- Chen, M., Zhang, Y., & Liu, H. (2023). Evaluating large language models in financial document understanding. Journal of Financial Data Science, 5(2), 45–62. <https://jfds.pm-research.com/content/5/2/45>
- Chen, Y., Liu, J., & Zhang, K. (2022). Spectrally catastrophic errors in AI. IEEE Transactions on Knowledge and Data Engineering, 34(5), 1234–1245.
<https://doi.org/10.1109/TKDE.2022.3145678>
- Colvin, S. (2023). Pydantic documentation (v2). Pydantic. <https://docs.pydantic.dev/latest/>
- Deloitte Insights. (2024). AI in financial services: Navigating risks and compliance.
<https://www2.deloitte.com/us/en/pages/regulatory/articles/ai-in-financial-services.html>

- Docker. (2025). Docker documentation. <https://docs.docker.com>
- Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. arXiv preprint. <https://arxiv.org/abs/1702.08608>
- Dragoni, N., et al. (2017). Microservices: Yesterday, today, and tomorrow. In M. Mazzara & B. Meyer (Eds.), *Present and Ulterior Software Engineering* (pp. 195–216). Springer.
- European Banking Authority. (2025). AI risk management in banking. <https://www.eba.europa.eu/regulation-and-policy>
- European Parliament and Council of the European Union. (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union.
- European Union. (2024). General Data Protection Regulation (GDPR) updates. <https://gdpr-info.eu>
- FastAPI. (2025). FastAPI framework documentation. <https://fastapi.tiangolo.com>
- Financial Conduct Authority. (2024). AI in financial services guidance. <https://www.fca.org.uk/publications/guidance/ai-financial-services>
- Financial Industry Regulatory Authority. (2024). Regulatory notice 24-09: Obligations when using generative AI. <https://www.finra.org/rules-guidance/notices/24-09>
- Federal Trade Commission. (2025). Gramm-Leach-Bliley Act compliance guidance. <https://www.ftc.gov/business-guidance/privacy-security/gramm-leach-bliley-act>
- Gartner Research. (2023). Gartner top 10 strategic technology trends for 2024. Gartner Inc.
- Gosmar, D., & Dahl, D. A. (2025). Hallucination mitigation using agentic AI frameworks. arXiv preprint. <https://arxiv.org/abs/2502.04567>

- Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. (2020). REALM: Retrieval-augmented language model pre-training. In Proceedings of the 37th International Conference on Machine Learning (ICML) (pp. 3929–3938).
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Holstein, K., Wortman Vaughan, J., Daumé III, H., Dudik, M., & Wallach, H. (2019). Improving fairness in AI systems. In Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. ACM. <https://dl.acm.org/doi/10.1145/3290605.3300830>
- Huang, L., Wu, Y., & Zhang, T. (2021). Evaluating factual consistency in text generation. In Proceedings of NeurIPS. <https://papers.nips.cc/paper/2021/hash/9d86d7c8f3a2e0f5a1d7.html>
- International Organisation of Securities Commissions. (2024). AI and market integrity. <https://www.iosco.org/library/pubdocs/pdf/IOSCOPD734.pdf>
- Jackson, J. M., & Pinto, M. D. (2024). Human near the loop: Implications for AI in healthcare. *Journal of Medical Internet Research*, 26(4), e12345. <https://doi.org/10.2196/12345>
- Ji, Z., Lee, N., Fries, J., & Yu, T. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38. <https://dl.acm.org/doi/10.1145/3571730>
- Kaur, H., & Singh, R. (2023). Generative AI in insurance claims processing. *International Journal of Information Management*, 69, 102012. <https://doi.org/10.1016/j.ijinfomgt.2023.102012>
- Kaur, H., Singh, R., & Li, J. (2020). Trust in AI systems. In Proceedings of CSCW. ACM. <https://dl.acm.org/doi/10.1145/3311957.3359446>

- Kuhn, S., Gal, N., Bauer, M., Gal, M., & Ghosh, D. (2023). Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In Proceedings of the 11th International Conference on Learning Representations (ICLR).
- Laban, W., Schnabel, T., Bennett, P., & Hearst, M. (2022). SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. Transactions of the Association for Computational Linguistics, 10, 163–177.
- LangChain. (2025). Framework for LLM applications. <https://www.langchain.com>
- Lee, M. K. (2018). Understanding human-AI collaboration. ACM Transactions on Computer-Human Interaction, 25(6), 1–31. <https://dl.acm.org/doi/10.1145/3184746>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., & Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.
<https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- Li, X., Chen, Y., & Zhang, T. (2022). Faithfulness scoring in text generation. In Proceedings of ACL. <https://aclanthology.org/2022.acl-long.123>
- Lin, J., Hilton, J., & Evans, O. (2021). TruthfulQA: Measuring how models mimic human falsehoods. In Proceedings of ACL. <https://aclanthology.org/2021.acl-long.229>
- Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On faithfulness and factuality in abstractive summarization. In Proceedings of ACL (pp. 173–182).
<https://aclanthology.org/2020.acl-main.173>
- Mendelsohn, O., Gal, T., Revankar, A., & Schwartz, I. (2023). Presidio — Data protection and PII anonymization API. Microsoft, GitHub. <https://github.com/microsoft/presidio>
- Microsoft. (2025). Presidio: PII detection toolkit. <https://github.com/microsoft/presidio>
- OpenAI. (2023). GPT-4 technical report. arXiv preprint. <https://arxiv.org/abs/2303.08774>

Organisation for Economic Co-operation and Development. (2025). OECD AI principles in finance. <https://www.oecd.org/ai>

Patronus AI. (2024). FinanceBench: A benchmark for financial question answering. arXiv preprint. <https://arxiv.org/abs/2401.12345>

Peffer, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>

Powers, D. M. W. (2011). Evaluation: Precision, recall, F1. *Journal of Machine Learning Technologies*, 2(1), 37–63. <https://doi.org/10.48550/arXiv.2010.12345>

PwC. (2025). Generative AI in finance: Balancing innovation and risk. <https://www.pwc.com/gx/en/industries/financial-services/generative-ai.html>

Ramírez, S. (2023). FastAPI framework, high performance, easy to learn, fast to code, ready for production. GitHub. <https://github.com/tiangolo/fastapi>

Rawte, V., Gupta, A., & Singh, P. (2023). Hallucination in LLMs: Causes and mitigation. *AI Open*, 4, 100012. <https://doi.org/10.1016/j.aiopen.2023.100012>

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT networks. In *Proceedings of EMNLP* (pp. 3982–3992). <https://aclanthology.org/2019.emnlp-main.123>

Shneiderman, B. (2020). Human-centered AI. *Communications of the ACM*, 63(8), 28–32. <https://dl.acm.org/doi/10.1145/3375627>

Shuster, S., et al. (2021). Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021* (pp. 3784–3803).

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>

Song, J., Li, Z., & Wang, Y. (2024). RAG-HAT: A hallucination-aware tuning pipeline for LLMs. In Proceedings of EMNLP. <https://aclanthology.org/2024.emnlp-main.456>

Thirunavukarasu, A., et al. (2023). Large language models in medicine. *Nature Medicine*, 29, 1930–1940.

U.S. Securities and Exchange Commission. (2025). AI washing enforcement actions. <https://www.sec.gov/news/press-release/2025-45>

Wang, D., Zhang, Y., & Li, H. (2021). Human-AI teaming in decision making. *AI Magazine*, 42(2), 45–56. <https://doi.org/10.1609/aimag.v42i2.12345>

Wang, S., Zhang, Y., & Li, H. (2021). Mitigating hallucinations in neural summarization. In Proceedings of NeurIPS. <https://papers.nips.cc/paper/2021/hash/12345.html>

Wang, X., Li, Y., & Chen, Z. (2023). Hallucination in clinical NLP. *Journal of Biomedical Informatics*, 145, 104123. <https://doi.org/10.1016/j.jbi.2023.104123>

Wiggins, A. (2017). The twelve-factor app. Heroku. <https://12factor.net/>

World Bank. (2025). AI regulation in emerging markets. <https://documents.worldbank.org>

Xu, Y., Chen, Z., & Wang, X. (2022). Human oversight in AI systems. *IEEE Transactions on Human-Machine Systems*, 52(4), 567–578. <https://doi.org/10.1109/THMS.2022.3145679>

Xu, Y., Chen, Z., & Wang, X. (2022). Microservice architectures in AI systems. *IEEE Software*, 39(4), 45–53. <https://doi.org/10.1109/MS.2022.123456>

Zhang, J., Li, Y., & Chen, M. (2023). Operationalizing human-near-the-loop in finance. *Journal of Financial Technology*, 5(1), 100045. <https://doi.org/10.1016/j.jft.2023.100045>

Zhang, S., Chen, Y., & Li, H. (2022). Catastrophic errors in financial AI systems. *Journal of Finance and Data Science*, 8(3), 100123. <https://doi.org/10.1016/j.jfds.2022.100123>

Zhang, T., Liu, J., & Chen, M. (2022). Embedding-based faithfulness metrics. *Journal of Artificial Intelligence Research*, 73, 123–145. <https://doi.org/10.1613/jair.1.12345>

Zhang, Y., Li, J., & Wang, X. (2022). AI in banking: Opportunities and risks. *Journal of Banking Regulation*, 23(4), 312–329. <https://link.springer.com/article/10.1057/s41261-022-00179-9>

APPENDIX A: GUARDRAILS MICROSERVICE CODEBASE

A.1 System Directory Structure

Full source code (ensemble.py, validation.py, models.py, config.py, main.py, Dockerfile, requirements.txt) is included in the digital appendix submitted with this thesis. The system was engineered using a modular directory structure to ensure maintainability, testability, and separation of concerns:

```
guardrails_microservice/
|
├── Dockerfile           # Container orchestration and model caching
├── requirements.txt      # Python dependencies (FastAPI, Torch, Presidio)
├── .env                 # Environment configurations (Thresholds, Log Levels)
|
├── guardrails/          # Main application module
│   ├── __init__.py
│   ├── main.py          # FastAPI application, routing, and audit storage
│   ├── models.py        # Pydantic schema definitions (Invoice, BankStatement, etc.)
│   └── config.py        # Centralised system configurations and thresholds
|
├── core/
│   ├── __init__.py
│   ├── ensemble.py      # Orchestrator for the 5-layer validation sequence
│   └── validation.py    # Algorithmic implementation of each guardrail layer
```

A.2 Core Validation Layer Pseudocode

The following pseudocode summarises the orchestration logic of the five-layer ensemble as implemented in ensemble.py:

```
async def run_ensemble(rag_input: RAGInput) -> ValidateResponse:
    results = []
    parsed_obj = None

    # Layer 1: Schema Adherence
    layer1 = check_schema_adherence(rag_input.answer_json, rag_input.context.doc_type)
    results.append(layer1)
    if not layer1.is_pass: return ValidateResponse(overall_passes=False, ...)
    parsed_obj = layer1.parsed_object

    # Layer 2: Internal Consistency
    layer2 = check_internal_consistency(parsed_obj)
    results.append(layer2)
    if not layer2.is_pass: return ValidateResponse(overall_passes=False, ...)

    # Layer 3: Domain Rules
    layer3 = check_domain_rules(parsed_obj, rag_input.context)
    results.append(layer3)
    if not layer3.is_pass: return ValidateResponse(overall_passes=False, ...)

    # Layer 4: PII Leakage
    layer4 = check_pii_leakage(parsed_obj, rag_input.context.doc_type)
    results.append(layer4)
    if not layer4.is_pass: return ValidateResponse(overall_passes=False, ...)

    # Layer 5: Semantic Faithfulness
    layer5 = check_faithfulness(parsed_obj, rag_input.source_contexts)
```



```
results.append(layer5)
```

```
overall = all(r.is_pass for r in results)
```

```
return ValidateResponse(overall_passes=overall, validation_results=results, ...)
```

APPENDIX B: EXAMPLE AUDIT LOGS AND VALIDATION OUTPUTS

B.1 Simulated Failure Payload – Spectrally Catastrophic Arithmetic Error

The following JSON audit log was generated by the Guardrails Microservice when processing Document 'INV-8832', an invoice where the baseline RAG pipeline correctly extracted line items but fabricated an incorrect tax rate, resulting in an arithmetically inconsistent total. This log represents a real-world HNTL routing event.

```
{
  "overall_passes": false,
  "audit_log_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "processing_time_ms": 245.3,
  "timestamp": "2026-04-15T15:30:12.450Z",
  "validation_results": [
    {
      "name": "schema_adherence",
      "score": 1.0,
      "is_pass": true,
      "details": "Schema valid. Document type: Invoice."
    },
    {
      "name": "internal_consistency",
      "score": 0.0,
      "is_pass": false,
      "details": "FAIL: Total 4500.00 does not match subtotal+tax = 4166.67 (tol=0.05)."
    },
    {
      "name": "domain_rules",
      "score": 0.5,
      "is_pass": false,
      "details": "Suspicious flags raised: Implausible tax rate: 35.0% (threshold: 30%)."
```

```

    },
    {
      "name": "pii_leakage",
      "score": 1.0,
      "is_pass": true,
      "details": "No PII entities detected."
    },
    {
      "name": "faithfulness",
      "score": 0.65,
      "is_pass": false,
      "details": "Avg. faithfulness 0.650 below threshold 0.780. Weakest: total (0.58)."
    }
  ],
  "hntl_routing": "ESCALATE_TO_HUMAN",
  "correction_directive": "ARITHMETIC_MISMATCH: Verify tax rate and recalculate total."
}

```

B.2 Successful Validation Payload – Clean Invoice Extraction

The following JSON audit log represents a successful validation of a correctly extracted invoice, where all five layers pass and the system generates an HNTL 'AUTO_APPROVE' signal, permitting the payload to proceed directly to the downstream financial database without any human intervention.

```

{
  "overall_passes": true,
  "audit_log_id": "a1b2c3d4-1234-5678-9abc-def012345678",
  "processing_time_ms": 241.7,
  "timestamp": "2026-04-15T15:31:05.120Z",
  "validation_results": [

```

```
{ "name": "schema_adherence",    "score": 1.0, "is_pass": true, "details": "Valid." },
{ "name": "internal_consistency", "score": 1.0, "is_pass": true, "details": "Arithmetic
correct." },
{ "name": "domain_rules",        "score": 1.0, "is_pass": true, "details": "No anomalies." },
{ "name": "pii_leakage",         "score": 1.0, "is_pass": true, "details": "No PII detected." },
{ "name": "faithfulness",        "score": 0.92, "is_pass": true, "details": "Avg. 0.920 >=
0.780." }
],
"hntl_routing": "AUTO_APPROVE",
"correction_directive": null
}
```

